UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA DE TECNOLOGÍAS Y SERVICIOS DE TELECOMUNICAIONES

TRABAJO FIN DE GRADO

Evaluación del protocolo QUIC en un balanceador de cargas para casos de estudio en 5G

ANDRÉS ROQUES SÁNCHEZ

2024

GRADO EN INGENIERÍA DE TECONOLGÍAS Y SERVICIOS DE TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

Título: Evaluación de los planos de datos programables y su aplicación a entornos 5G

Autor: D. Andrés Roques Sánchez

Tutor: D. José Ignacio Moreno Novella

Cotutor: D. Jorge Andrés Brito Collantes

Ponente: D. ……..

Departamento: Departamento de Ingeniería de Sistemas Telemáticos (DIT)

MIEMBROS DEL TRIBUNAL

Presidente: D. ……..

Vocal: D. ……..

Secretario: D. ……..

Suplente: D. ……..

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de: ……

Madrid, a de de 20…

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA DE TECNOLOGÍAS Y SERVICIOS DE TELECOMUNICACIÓN

TRABAJO FIN DE GRADO

Evaluación del protocolo QUIC en un balanceador de cargas para casos de estudio en 5G

ANDRÉS ROQUES SÁNCHEZ

2024

RESUMEN

El impacto de las implementaciones de 5G en los últimos años es evidente, convirtiéndose en prioridad el estudio y la mejora de las redes que las soportan. Se requiere tecnologías innovadoras que mejoren la capacidad, flexibilidad y agilidad de la red para continuar su desarrollo. En este contexto, surge el protocolo QUIC como una alternativa prometedora a TCP. Desarrollado inicialmente por Google y estandarizado por el IETF, QUIC se caracteriza por mejorar la eficiencia y la seguridad de las conexiones web. Estas propiedades pueden resultar especialmente beneficiosas en entornos de redes móviles de alta velocidad como 5G.

Este trabajo se centra en la evaluación del protocolo QUIC en un balanceador de carga diseñado para posibles casos de estudio en arquitecturas 5G, utilizando las funcionalidades del plano de datos programable implementadas por el lenguaje de programación P4. Estas tecnologías habilitantes permiten realizar pruebas y análisis detallados, facilitando la implementación y comparación de QUIC frente a TCP, que ha demostrado ser robusto y confiable, pero que presenta varias limitaciones.

Además de la evaluación técnica, el trabajo también considera aspectos prácticos como la implementación y la compatibilidad con aplicaciones existentes, así como los posibles desafíos y limitaciones asociados con la adopción de QUIC en un entorno de producción. Se espera que los resultados de este estudio proporcionen información valiosa para operadores de redes, proveedores de servicios y desarrolladores de aplicaciones que buscan mejorar el rendimiento y la eficiencia de sus sistemas en el contexto de las redes 5G.

PALABRAS CLAVE

5G, plano de datos programable, P4, QUIC, UDP, TCP, balanceador de carga.

ABSTRACT

The recent surge in 5G network adoption has intensified research efforts towards enhancing network capabilities. This necessitates the exploration of innovative technologies that can augment network capacity, flexibility, and agility to support continued advancements. In this context, the QUIC protocol, initially developed by Google and subsequently standardized by the IETF, emerges as a promising alternative to the well-established TCP protocol. QUIC offers potential improvements in terms of efficiency and security for web connections, particularly within high-speed mobile network environments like 5G.

This study focuses on evaluating the performance of the QUIC protocol within a load balancer designed for 5G network architectures. The evaluation leverages the benefits and functionalities provided by the programmable data plane and the P4 programming language. These enabling technologies facilitate comprehensive testing and analysis, offering a robust and adaptable platform for implementing and comparing QUIC against the established and reliable, yet potentially limited, TCP protocol.

Beyond the technical evaluation, this research also delves into practical considerations such as implementation complexities, compatibility with existing applications, and potential challenges associated with deploying QUIC in real-world production environments. The anticipated outcome of this study is to generate valuable insights that can inform network operators, service providers, and application developers in their endeavors to optimize the performance and efficiency of their systems within the context of 5G networks.

KEYWORDS

5G, programmable data plane, P4, QUIC, UDP, TCP, load balancer.

AGRADECIMIENTOS

Quiero dar las gracias a todas las personas que me han ayudado a lo largo de todos estos últimos años. Especial mención hacia mis padres, Inma y Luis y a mis hermanas Irene y Ángela por haberme apoyado siempre y favorecer a mi desarrollo como persona dentro de casa.

Gracias también a mis abuelos Paco y Paquita y a mi tío Jose, por haber sido una pieza fundamental durante toda mi vida y haber emprendido la labor de padres tantas veces cuando ha sido necesario.

Gracias a mis abuelos Pipe y Sarah, que pese a la distancia siempre me han mostrado su apoyo, su confianza y su orgullo.

Gracias a toda mi familia restante, tíos, primos, sobrinos, padrinos, etc. por haberme ayudado a ser quien soy en todo momento.

Gracias a todos los amigos por ser esos compañeros de vida elegidos, que por suerte, parece ser que elijo de maravilla, por no solo una vía de disfrute sino también de calma y entendimiento.

Gracias a mis tutores, José Ignacio, por confiar en mí desde el principio y darme la oportunidad de realizar el Trabajo Fin de Grado en el departamento y Jorge Brito por tener la paciencia suficiente conmigo para encomendarme este proyecto y saber guiarme de una manera tan precisa.

Gracias a los maestros y profesores que he tenido a lo largo de mi vida, ya que han sabido dejar huella en mí y despertar esa curiosidad, esas ganas de aprender y de mejorar.

Por último, y no menos importante (aunque un poco ególatra), agradecerme a mí mismo por el esfuerzo y trabajo realizado hasta llegar aquí, del que nadie mas yo mismo es consciente y esa autovaloración me ha llevado hasta completar este recorrido como es este Grado en Ingeniería de Tecnologías y Servicios de Telecomunicación.

INDICE

RESUMEN ..... **I**
ABSTRACT ..... **II**
AGRADECIMIENTOS ..... **III**
LISTA DE ACRÓNIMOS ..... **VIII**
1. INTRODUCCIÓN Y OBJETIVOS **1**
1.1. Introducción 1
1.2. Objetivos 1
1.3. Medios materiales 2
1.4. Estructura del documento 2
2. ESTADO DEL ARTE **3**
2.1. SDN, redes virtuales 3
2.2. Planos funcionales de los nodos de red 4
2.2.1. Programabilidad del plano de datos 5
2.3. Arquitectura PISA 6
2.4. P4 y sus utilidades 7
2.5. Protocolo QUIC 8
2.5.1. Servicios HTTP y contexto de QUIC 9
2.5.2. QUIC vs TCP 11
2.5.3. Formato de los paquetes QUIC 12
2.5.4. Balanceo de carga e implementación con QUIC 13
2.6. Tecnología y redes 5G 15
2.6.1. Arquitecturas 16
2.6.2. Aplicaciones de planos de datos programableS A 5G 18
3. DESARROLLO Y RESULTADOS **19**
3.1. Instalación de herramientas PARA AMBIENTE P4 19
3.2. Primer escenario, Prueba de TCP 20
3.2.1. Explicacion de las pruebas a realizar 20
3.2.2. Resultados de rendimiento 24
3.3. Segundo escenario, Prueba QUIC 31
3.3.1. Explicación de las pruebas a realizar 31
3.3.2. Resultados de rendimiento 36
3.4. Comparación de los escenarios estudiados 41
3.5. Posibles aplicaciones en 5G 42

4. CONCLUSIONES Y LÍNEAS FUTURAS 43

4.1. Conclusiones 43

4.2. Líneas futuras 44

5. BIBLIOGRAFÍA 45

ANEXO A: CÓDIGOS COMPLETOS DE LOS ESCENARIOS 49

A.1 Códigos el escenario 1 49

A.2 Códigos el escenario 2 56

ANEXO B: ASPECTOS ÉTICOS, ECONÓMICOS, SOCIALES Y AMBIENTALES 67

B.1 Introducción 67

B.2 Descripción de impactos relevantes relacionados con el proyecto 67

B.3 Análisis detallado de alguno de los principales impactos 67

B.4 Conclusiones 68

ANEXO C: PRESUPUESTO ECONÓMICO 69

INDICE DE FIGURAS Y TABLAS

Figura 1. Comparación de las redes tradicionales y SDN. [1] 3
Figura 2. Representación de los planos funcionales. [5] 4
Figura 3. Representación geométrica de los planos funcionales. [6] 5
Figura 4. Liberación de las capas de control y de datos. [6] 5
Figura 5. Liberación del plano de datos [3] 5
Figura 6. Estructura PISA. [3] 6
Figura 7. Representación de una arquitectura empleando P4. [1] 8
Figura 8. Establecimiento de conexión con QUIC para a) la primera conexión y b) subsecuentes conexiones 9
Figura 9. Cronología del despliegue de HTTP. [13] 10
Figura 10. Despliegue de HTTP/2 y HTTP/3 en mayo de 2021. [13] 10
Figura 11. Uso de HTTP/3 actual [14]. 10
Figura 12. Uso de HTTP/2 actual [15]. 10
Figura 13. Empleo de QUIC en la conectividad HTTP. [17] 11
Figura 14. Cabecera de un paquete en QUIC. [13] 12
Figura 15. Red con balanceador de carga *Round Robin*. [22] 13
Figura 16. Casos de uso de las comunicaciones con 5G. [25] 15
Figura 17. Evolución de las redes. Operadora DIGI [28] 16
Figura 18. Diagrama de SBA [33] 17
Figura 19. Topología del escenario 1. 21
Figura 20. Representación de la carpeta con los archivos necesarios para el escenario 1 21
Figura 21. Representación de la máquina de estados del *Parser* TCP 23
Figura 22. Generación de Mininet del escenario 26
Figura 23. Agregación de entradas a las tablas de sw2 27
Figura 24. Comando para comprobar conexión ICMP 27
Figura 25. Envío de la conexión TCP 28
Figura 26. Recepción de la conexión TCP 28
Figura 27. Visualización de los paquetes circulados en el enlace h1-s1 28
Figura 28. Trazas de la configuración P4 en sw1 tras la conexión ICMP 28
Figura 29. Trazas de la configuración P4 en sw1 tras la conexión TCP 29
Figura 30. Comando para iniciar una conexión de 1000 peticiones desde H1 hacia H1 29
Figura 31. Resultado de observar el enlace s1-s2 30
Figura 32. Resultado de observar el enlace s1-s3 30
Figura 33. Resultado de observar el enlace s1-s4 30

Figura 34. Resultado de observar el enlace s1-s5 30
Figura 35. Repartición del tráfico por cada ruta del escenario: 31
Figura 36. Topología del escenario 2. 32
Figura 37. Representación de la carpeta principal para el escenario 2 33
Figura 38. Representación de la carpeta picoquic para el escenario 2 33
Figura 39. Representación de la máquina de estados del *Parser* QUIC 34
Figura 40. Creación correcta de la Mininet del escenario 2 37
Figura 41. Configuración de las tablas del *switch* del escenario 2 38
Figura 42. Captura de las solicitudes a los servidores. 38
Figura 43. Trazas del *parser* del *switch* 39
Figura 44. Pruebas de *ping* entre los elementos de la red 39
Figura 45. Configuración de cliente solicitante en el *host* 40
Figura 46. Trazas de las solicitudes UDP/QUIC del cliente 40

LISTA DE ACRÓNIMOS

AF: Application Function / Función de Aplicación

AMF: Access and Mobility Management Function / Función de Gestión de Acceso y Movilidad

API: Application Programming Interface / Interfaz de Programación de Aplicaciones

AUSF: Authentication Server Function / Función del Servidor de Autenticación

CPD: Control Plane Function / Función del Plano de Control

DDoS: Distributed Denial of Service / Denegación de Servicio Distribuido

DN: Data Network / Red de Datos

EPC: Evolved Packet Core / Núcleo de Paquetes Evolucionado

FEC: Forward Error Correction / Corrección de Errores hacia Adelante

FN: Function / Función

HTTP: Hypertext Transfer Protocol / Protocolo de Transferencia de Hipertexto

HTTP/N: HTTP version N / Versión N de HTTP

HTTPs: HTTP Secure / HTTP Seguro

ICMP: Internet Control Message Protocol / Protocolo de Mensajes de Control de Internet

INT: Intent-Based Networking / Redes Basadas en Intenciones

IP: Internet Protocol / Protocolo de Internet

IPvN: IP version N / Versión N de IP

ITU: International Telecommunication Union / Unión Internacional de Telecomunicaciones

LB: Load Balancer / Balanceador de Carga

LPM: Longest Prefix Match / Coincidencia de Prefijo Más Largo

LTE: Long-Term Evolution / Evolución a Largo Plazo

MAC: Media Access Control / Control de Acceso al Medio

MATs: Manufacturing and Automation Testing Standards / Estándares de Pruebas deManufactura y Automatización

MMTC: Massive Machine Type Communications / Comunicaciones Masivas de Tipo Máquina

MTU: Maximum Transmission Unit / Unidad máxima de transferencia

NAT: Network Address Translation / Traducción de Dirección de Red

NGAP: Next Generation Application Protocol / Protocolo de Aplicación de Próxima Generación

NSA: Non-Standalone / No Independiente

NR: New Radio / Nueva Radio

ONF: Open Networking Foundation / Fundación de Redes Abiertas

P4: Programming Protocol-Independent Packet Processors / Procesadores de Paquetes Programables Independientes de Protocolo

PCF: Policy Control Function / Función de Control de Políticas

PFCP: Packet Forwarding Control Protocol / Protocolo de Control de Reenvío de Paquetes

PISA: Platform for International Student Assessment / Plataforma para la Evaluación Internacional de Estudiantes

PtP: Point-to-Point / Punto a Punto

QUIC: Quick UDP Internet Connections / Conexiones Rápidas de Internet sobre UDP

RTT: Round-Trip Time / Tiempo de ida y vuelta

SBA: Service-Based Architecture / Arquitectura Basada en Servicios

SDN: Software-Defined Networking / Redes Definidas por Software

SMF: Session Management Function / Función de Gestión de Sesiones

SSH: Secure Shell / Shell Seguro

TLS: Transport Layer Security / Seguridad de la Capa de Transporte

SSL: Secure Sockets Layer / Capa de Conexión Segura

UDP: User Datagram Protocol / Protocolo de Datagramas de Usuario

UDM: Unified Data Management / Gestión Unificada de Datos

uRLLC: Ultra-Reliable Low Latency Communications / Comunicaciones Ultrarresilientes de Baja Latencia

UPF: User Plane Function / Función del Plano de Usuario

VIP: Virtual IP / IP Virtual

VM: Virtual Machine / Máquina Virtual

1. INTRODUCCIÓN Y OBJETIVOS

1.1. INTRODUCCIÓN

Desde 2019, cuando comenzaron a desplegarse las primeras redes 5G, la tecnología ha avanzado rápidamente y ha capturado la atención de expertos en telecomunicaciones. La red 5G promete velocidades de conexión significativamente más rápidas, una latencia mucho menor y la capacidad de conectar un número masivo de dispositivos simultáneamente. Sin embargo, la implementación completa de 5G aún está en progreso, con muchas redes operando en modo Non-Stand Alone (NSA), utilizando infraestructura de la generación anterior.

En este contexto de transición y evolución tecnológica, el protocolo QUIC se presenta como una solución innovadora para optimizar las comunicaciones en redes de próxima generación. Desarrollado por Google y estandarizado por la IETF, QUIC está diseñado para ofrecer mejoras sustanciales en la velocidad, la seguridad y la eficiencia de la transmisión de datos, operando sobre UDP y superando las limitaciones de los protocolos tradicionales como TCP.

El uso de QUIC en balanceadores de carga es una prominente opción a ser aplicada en entornos de 5G. Los balanceadores de carga son cruciales para distribuir el tráfico de red de manera equitativa entre múltiples servidores, asegurando una alta disponibilidad y una respuesta rápida. Con la integración de QUIC, se espera mejorar aún más el rendimiento de estos sistemas, permitiendo una gestión de tráfico más eficiente y resiliente, capaz de manejar la demanda creciente y los requisitos estrictos de latencia y velocidad.

Además, el plano de datos programable y el lenguaje de programación P4 se destacan como tecnologías habilitadoras clave para esta implementación. Fundamentándose en los principios de las Redes Definidas por Software (*Software-Defined Networking*, SDN), el plano de datos programable permite una mayor flexibilidad y control sobre la red, permitiendo a los desarrolladores definir cómo se debe procesar y manejar el tráfico. P4, específicamente, es un lenguaje diseñado para programar estos planos de datos, ofreciendo un alto grado de personalización y optimización.

Una implementación efectiva de QUIC, empleando las facilidades que aporta la programabilidad del plano de datos con P4, podría significar un avance significativo en la manera en que se gestionan y optimizan las redes, especialmente en un entorno 5G. Esta investigación se centrará en analizar cómo QUIC puede integrarse en balanceadores de carga, evaluando su rendimiento y ventajas en comparación con las soluciones actuales. Los resultados podrían ofrecer una visión clara de las mejoras potenciales en la infraestructura de redes, contribuyendo al desarrollo de una red más eficiente y robusta.

1.2. OBJETIVOS

El objetivo principal de este trabajo consiste en la implementación y evaluación del protocolo QUIC para posibles aplicaciones en arquitecturas 5G, empleando el plano de datos programable y el lenguaje de programación P4 como tecnologías habilitantes. Para lograr este objetivo, se establecen los siguientes subobjetivos:

1. ESTADO DEL ARTE. Se realizará una revisión de la tecnología actual que rige las SDN, el plano de datos programable , la arquitectura PISA, el lenguaje P4, así como los casos de uso para 5G que esta habilita. De igual forma, se incluyen los fundamentos teóricos detrás de QUIC y el protocolo de uso más tradicional, TCP.

2. EVALUACIÓN TECNOLÓGICA DE ARQUITECTURAS. Se estudiarán los posibles escenarios y arquitecturas para la implementación específica de balanceadores de cargas.

3. IMPLEMENTACIÓN DEL ESCENARIO. Se desplegarán arquitecturas SDN con el fin de implementar y comparar balanceadores de carga en P4 empleando los protocolos TCP y QUIC.

4. VALIDACIÓN. Se realizará una evaluación global de los escenarios desarrollados y se realizará una comparativa de los mismos.

5. CASOS DE USO. Una vez validada la evaluación del escenario planteado, se propondrán distintos casos de uso del protocolo QUIC para su posible implementación en arquitecturas 5G.

1.3. MEDIOS MATERIALES

Debido a la naturaleza de prueba de concepto del presente trabajo, se ha realizado un despliegue de una máquina virtual para el proceso de estudio, desarrollo e implementación. Este despliegue se ha llevado a cabo en un ordenador personal mediante el uso de Virtual Box. Por tanto, los materiales considerados para la utilización del trabajo son:

- Ordenador Lenovo IdeaPad con un procesador IntelCore i5-8265U con 4 procesadores lógicos.
- Máquina Virtual VirtualBox con sistema operativo Ubuntu 20.04.

1.4. ESTRUCTURA DEL DOCUMENTO

La memoria del presente trabajo consta de la siguiente arquitectura:

Capítulo 1: Introducción del proyecto, así como una descripción de los objetivos que se pretenden lograr con él y los medios materiales utilizados para su realización.

Capítulo 2: Revisión del estado del arte, revisando el punto en el que se encuentra la tecnología actual empleada en el plano de datos las SDN, arquitectura PISA, lenguaje de reenvío de paquetes programable P4 y los casos de uso para las arquitecturas 5G que esta permite. Así como los fundamentos teóricos de los protocolos TCP y QUIC.

Capítulo 3: Diseño de dos escenarios de aplicación de las tecnologías estudiadas en arquitecturas SDN con la implementación de balanceadores de carga programados con P4 empleando el protocolo TCP y QUIC.

Capítulo 4: Validación y evaluación global de los escenarios implementados y de sus posibles casos de uso aplicados a gran escala. Además de las posibles líneas futuras del proyecto.

Anexo A: Códigos empleados para el desarrollo de los escenarios

Anexo B: Aspectos éticos, económicos, sociales y ambientales

Anexo C: Presupuesto económico

2. ESTADO DEL ARTE

2.1. SDN, REDES VIRTUALES

Las SDN o *Software-Defined Networking*, como su nombre indica son redes virtuales definidas por software. Estas redes permiten el control de la red de la forma definida por el usuario y le da el cuerpo a la programabilidad del plano de control. [1]

La aparición de las SDN generó un cambio en el paradigma del entendimiento de las redes habituales, ya que se consiguió separar los planos funcionales de los nodos de la red: plano de datos, de control y de gestión, como vemos en la Figura 1. [2]

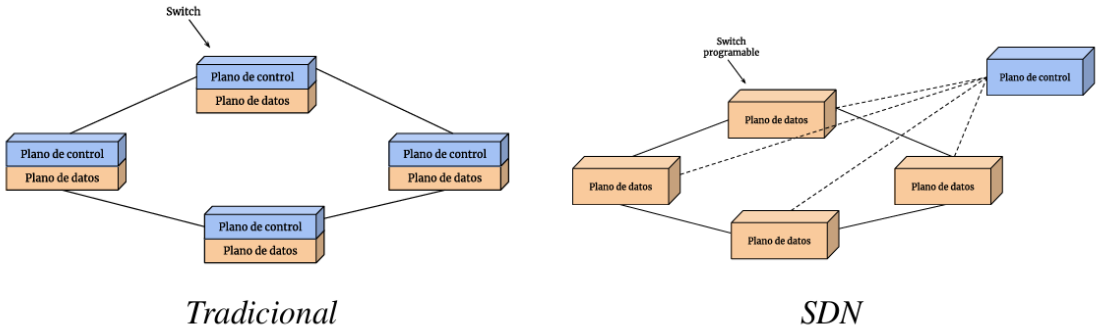


Figura 1. Comparación de las redes tradicionales y SDN. [1]

Los dispositivos de red como *routers* o *switches* tienen complejos algoritmos para el plano de control y el de datos. Debido a que en las redes tradicionales no se consigue separar los planos funcionales, estos algoritmos eran fijos para cada dispositivo y para ambos planos funcionales del mismo. Los algoritmos dependían únicamente de los vendedores de los dispositivos, quienes eran los que los fijaban a cada dispositivo previamente de su salida al mercado. [3]

Las SDN surgen como la primera posible solución para liberar los planos funcionales y lograr su programabilidad por el usuario, rompiendo con la inflexibilidad del dispositivo programado de manera fija por el vendedor.

Para algunos dispositivos, los vendedores permitieron a los usuarios introducir sus propios algoritmos del plano de control. Estos algoritmos ofrecían la información de reenvío necesaria al plano de datos que aún no era reemplazable en los dispositivos y se mantenía fijo a la elección del vendedor.

Para completar el nacimiento de las SDN se decidió proporcionar una API que pudiese ser llamada remotamente para el control de este plano. Esta API sería OpenFlow, encargada de estandarizar la mensajería del plano de control liberado. [4]

2.2. PLANOS FUNCIONALES DE LOS NODOS DE RED

Los dispositivos de red actuales constan con tres planos principales de funcionamiento: el plano de datos, el plano de control y el plano de gestión, representados en la Figura 2. [5]

El plano de datos. Consiste en las tareas que realiza un dispositivo de la red para reenviar todo el tráfico que fluye a través de este, llamado mecanismo de encaminamiento. Este se encarga del procesado de paquetes, donde los observa, comprueba que no poseen errores, y determina su manipulación; y tras ello, de su expedición hacia su destino final.

El plano de control. Se refiere a toda acción que dirija el plano de datos. La mayoría de estas acciones tienen que ver con la creación de tablas de enrutamiento o de direcciones y de sus protocolos de resolución.

El plano de gestión. Incluye protocolos que permiten a los ingenieros de red administrar los dispositivos. Este realiza trabajos generales, pero no impactan directamente en el plano de datos. Telnet y Secure Shell (SSH) son dos de los protocolos del plano de gestión más conocidos.

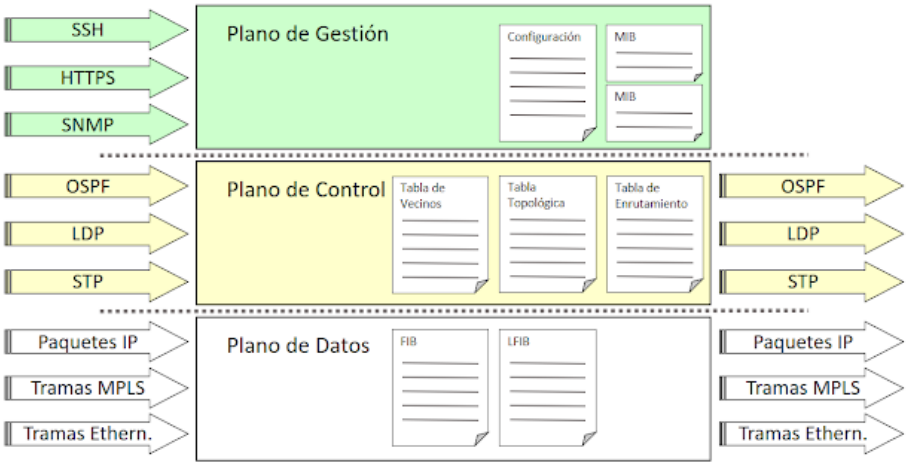


Figura 2. Representación de los planos funcionales. [5]

El plano de gestión no suele ser considerado, debido a que las funciones del plano de gestión no tienen impacto en los otros planos, al contrario que el de control y el de datos cuyas funciones están altamente relacionadas. Es decir, podemos considerar que el plano de control y el de datos trabajan en de forma paralela y el plano de gestión de forma perpendicular a ellos, como lo representa la Figura 3.

El plano de control y el de gestión, suelen ser considerados como una capa conjunta de control de datos. Para las SDN, donde hemos visto que se independizan los planos de control y de datos, estas capas se ven libres a su funcionamiento independiente, antes limitado a ser conjunto y programado estáticamente por el vendedor. Esto permite que, operando con el plano de datos de manera libre, los administradores de las redes pueden crear una infraestructura de tráfico de red más flexible, eficiente, ágil y escalable, con un mejor control y visibilidad sobre los datos de la red. Podemos verlo de forma gráfica en la Figura 4. [6]

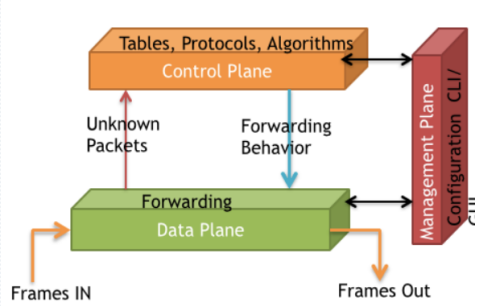


Figura 3. Representación geométrica de los planos funcionales. [6]

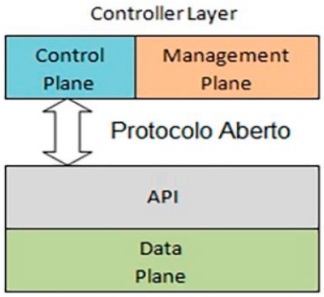


Figura 4. Liberación de las capas de control y de datos. [6]

2.2.1. PROGRAMABILIDAD DEL PLANO DE DATOS

Como se ha comentado anteriormente, con la inclusión de las SDN, el plano de control se libera del resto de planos funcionales fijos de los dispositivos de red, como observamos en la Figura 5, permitiendo una programación libre por parte del usuario. [3]

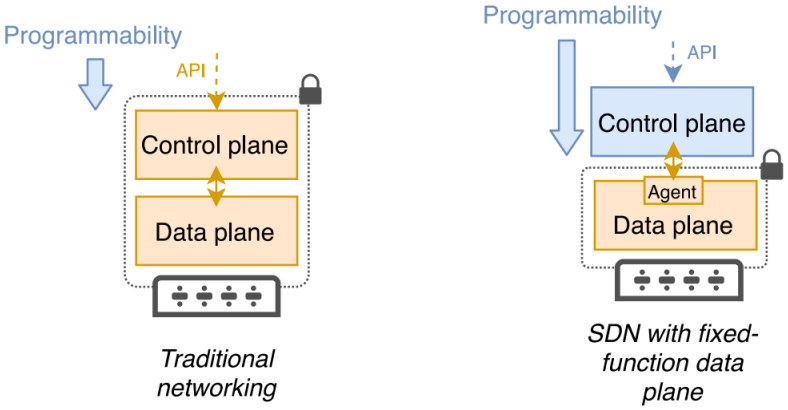


Figura 5. Liberación del plano de datos [3]

La programabilidad del plano de datos es un concepto que se refiere a la habilidad para particularizar y manipular el reenvío de datos que pasan por los dispositivos de la red. Esto se consigue programando el hardware del dispositivo encargado del plano de datos, o su software directamente.

El plano de datos es el responsable de la transferencia de los paquetes de datos a través de la red y es normalmente implementado en dispositivos de procesamiento de paquetes de gran velocidad. Tradicionalmente, el plano de datos ha sido manejado por el firmware de los dispositivos de la red, el cual es un software preinstalado por el fabricante, que proporciona instrucciones de máquina a los componentes de hardware de un dispositivo, lo que le permite funcionar a un nivel básico . Esto limitaba la capacidad para particularizar o adaptar dinámicamente estos dispositivos a las condiciones de la red.

Con la importancia que han tomado las SDN, la programabilidad del plano de datos se ha convertido en un ámbito muy relevante debido a que permite el despliegue de arquitecturas de redes muy dinámicas y flexibles que pueden responder a las necesidades de varios tipos de aplicaciones y servicios (ej., ciberseguridad, telemetría en tiempo real, optimización de tráfico, etc.).

La programabilidad del plano de datos se puede viabilizar a través del lenguaje P4 dentro de la arquitectura PISA de control de paquetes y la actividad de una API (*Application Programming Interface*), que pueda ser llamada por un controlador (plano de control) de la SDN.

Como veremos posteriormente, la liberación del plano de datos abre la veda a la utilización de elementos de red programables, que como veremos posteriormente, implementándolos en una arquitectura de 5G pueden generar un avance importante en términos de latencia, rendimiento, seguridad, etc. Mismos que son necesarios para el despliegue global de 5G y el alcance de sus requerimientos y capacidades.

2.3. ARQUITECTURA PISA

PISA (*Protocol-Independent Switch Architecture*) es un modelo de *switch* programable que habilita el control granular sobre los paquetes procesados cuya arquitectura se refleja en la Figura 6 [3] [7]. La arquitectura de este modelo se compone de los siguientes elementos:

- *Parser*. Permite a los programadores declarar cabeceras arbitrarias conjuntamente, empleando una máquina de estados finita que define el orden de las cabeceras dentro de los paquetes. Además, estructura idealmente las cabeceras de los paquetes.

- *Match-action pipeline*. Se trata de una consecución de varias unidades de *Match-Action* lógicas. Cada una de estas unidades incluye una o más MATs (*Match-Action Table*) cuya labor es identificar paquetes entrantes y asignarles una acción especifica, teniendo en cuenta las reglas de acción definidas según unos datos de acción otorgados.

- *Deparser*. Permite al programador definir cómo deben estructurarse las cabeceras de los paquetes para que se consideren serializados.

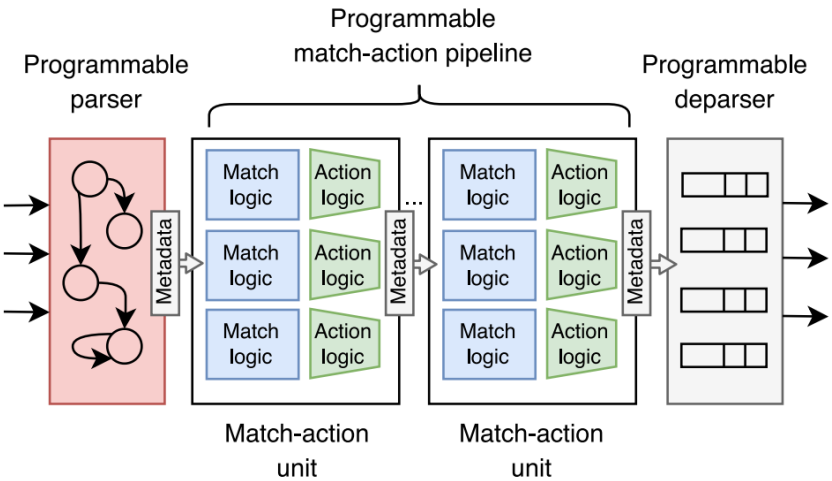


Figura 6. Estructura PISA. [3]

Un paquete procesado por el pipeline de PISA consta de la carga útil del paquete y sus metadatos. PISA sólo procesa los metadatos de los paquetes que viajan desde el *parser* hasta el *deparser*, la carga útil del paquete viaja por separado. Estos metadatos, que serán descartados una vez el paquete abandone el pipeline, se pueden dividir en:

- Cabeceras de los paquetes. Son los que corresponden al protocolo de red empleado. Suelen ser extraídos en el *parser* y emitidos en el *deparser* o en ambos.

- Definidos por el usuario. Es un registro de valor temporal que permite a los desarrolladores añadir información a los paquetes que pueden ser usados durante el procesado del pipeline.

- Metadatos intrínsecos. Provienen de los componentes de la red con el plano de control liberado, llamados *fixed-function* (observar Figura 5). Los componentes programables podrán recibir información de los compontes *fixed-function* leyendo estos metadatos intrínsecos, o controlar su comportamiento configurando los metadatos que consumen. Por ejemplo, para el envío de un paquete la unidad *match-action* genera un metadato intrínseco que representa un número de puerto de salida el cual será consumido por el controlador de tráfico y/o el bloqueador de puertos de salida.

Además de los componentes programables de PISA, las arquitecturas de *switch* suelen incluir otros componentes *fixed-function*, como bloqueadores de puertos de entrada/salida que reciben o mandan paquetes, motores de réplicas de paquetes o controladores de tráfico.

2.4. P4 Y SUS UTILIDADES

P4 (*Programming Protocol-Independent Packet Processors*) [8] es el lenguaje principal de programación para el plano de datos en arquitecturas PISA. La primera idea del lenguaje surgió en el 2013 y fue publicada oficialmente en un artículo de Bosshart et al. en 2014 [9] . Sin embargo, no sería hasta 2018 cuando surgió la primera versión estandarizada del lenguaje, $P4_{14}$ que continuaría desarrollándose hasta la versión actual que es $P4_{16}$ (2020). Las versiones del lenguaje son revisadas y aceptadas por el "P4 Language Consortium" que desde 2019 hasta principios de 2024 formaba parte del ONF (*Open Networking Foundation*). Sin embargo, en enero de 2024, P4 se consagró como un proyecto independiente, uniendo las investigaciones pertenecientes a la ONF a la LF (Linux Foundation) que albergará su porfolio de proyectos [10]. Este consorcio de P4 es dirigido por un comité técnico y consta de cinco grupos de trabajo.

El lenguaje P4 tiene una estructura similar a C y sus programas se dividen en 4 secciones, una parte para la declaración de los datos (cabeceras) y otra para cada sección de PISA:

1. La primera consiste en la de declaración de datos, donde se crea un objeto de cada una de las cabeceras a identificar, fijando los campos de cada una de ellas y se define un objeto cabecera que se usará en el resto del código.

2. La segunda se basa en la lógica del *parser*, en ella se extrae la cabecera a modificar y se guardan sus campos en el objeto cabecera creado anteriormente.

3. La tercera sección se encarga de programar las tablas *Match & Action*, escribiendo ahí los cambios que se han de realizar en la cabecera seleccionada.

4. Por último, la cuarta parte del código es la responsable del *deparser* en la que se modifica finalmente la cabecera.

P4 es el lenguaje utilizado en la gran mayoría de las implementaciones PISA en planos de datos programables. No obstante, existen otras opciones de lenguajes para arquitecturas PISA como son: FAST, OpenState, Domino, FlowBlaze, ProtocolOblivious, Forwarding and NetKAT. Además de estos lenguajes, Broadcom y Xilinx ofrecen planos de datos programables específicos para el vendedor basados en tablas *match-action*.

Como cualquier lenguaje no interpretado, P4 necesita un compilador aparte para que el código pueda ser entendido por el resto de los elementos del sistema. Este compilador tiene la tarea de interpretar el programa, entender su funcionalidad y comunicarle esta información al plano de datos y el plano de control simultáneamente, consiguiendo una comunicación entre ellos mediante una API a tiempo real. En la Figura 7 vemos un escenario de uso P4 en una escala global de una red SDN.

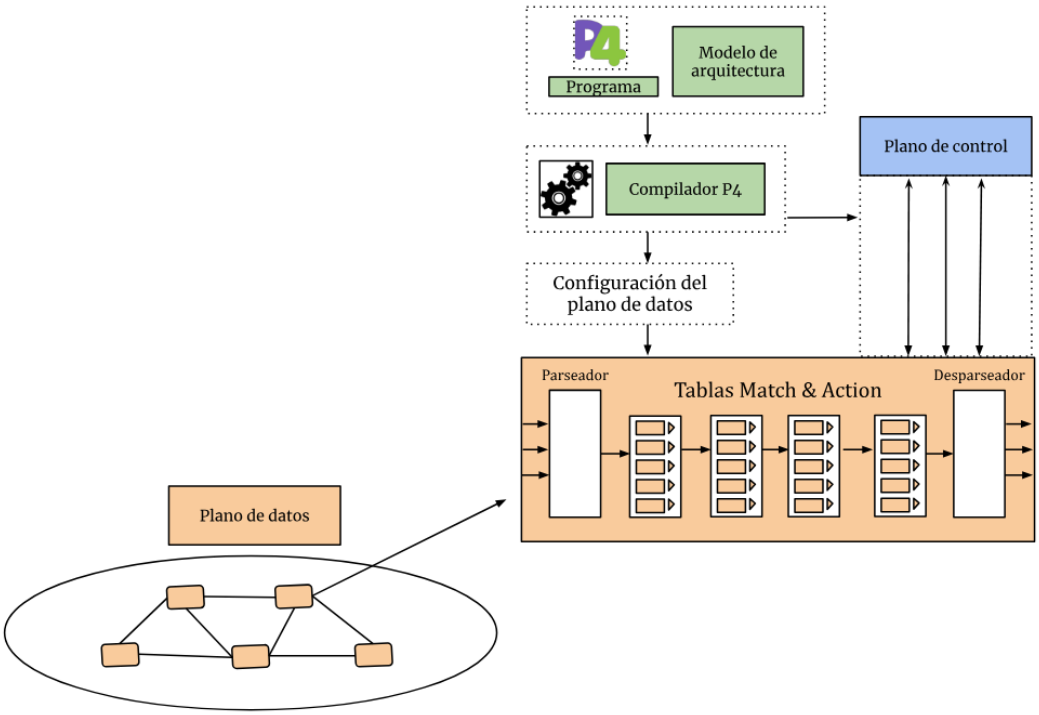


Figura 7. Representación de una arquitectura empleando P4. [1]

2.5. PROTOCOLO QUIC

QUIC es un protocolo de transporte experimental de Google publicado en 2013. El nombre se deriva de "*Quick UDP Internet Connections*", que se debe a que este posibilita el envío de paquetes sencillos de datos a través del UDP (*User Datagram Protocol*), que no necesita conexión. El telón de fondo de los trabajos en QUIC fue el deseo de desarrollar una alternativa a las soluciones de seguridad establecidas principalmente a partir de TCP (Protocolo de Control de Transmisión) pero también de HTTP/2 y TLS/SSL (*Transport Layer Security / Secure Sockets Layer*) que ofrecen la misma protección, pero también una demora reducida en la conexión y en el transporte, y posibilita conexiones multiplexadas. [11]

Con este motivo, Google ha diseñado QUIC de manera que el mismo protocolo regule el control de la conexión. En el primer acuerdo de conexión (*handshake*), reflejado en la Figura 8, entre emisor y receptor, estos se intercambian los certificados y las claves necesarios para el cifrado de los datagramas enviados. Este intercambio no se aplica en posteriores comunicaciones, lo que reduce la latencia. Como protocolo de cifrado se aplica la actual versión TLS 1.3 con velocidad optimizada (versión estandarizada en marzo de 2017), que obtuvo preferencia a la solución criptográfica propia. [12]

En términos de multiplexación, QUIC se orienta al protocolo SPDY elaborado por Google, que sirvió de modelo para HTTP/2: los flujos de datos se envían a través de una única conexión cliente-servidor, por lo que disminuye el tiempo de carga.

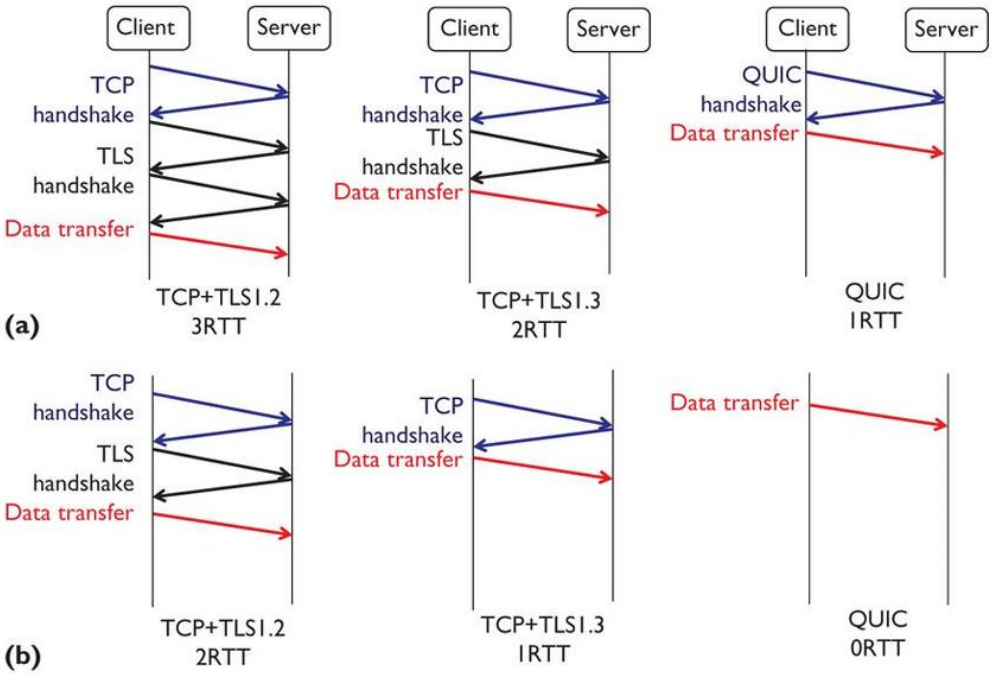


Figura 8. Establecimiento de conexión con QUIC para a) la primera conexión y b) subsecuentes conexiones.

Las conexiones QUIC permanecen abiertas hasta que el servidor decide terminarlas sin notificar al cliente la decisión. Esto puede ocurrir por dos razones: uno de los extremos envía un paquete "CONNECTION_CLOSE" a su par, lo cual se denomina apagado explícito; o por inactividad de la conexión, llamado apagado implícito, cuyo tiempo por defecto es de 30 segundos, configurable hasta un máximo de 10 minutos.

2.5.1. SERVICIOS HTTP Y CONTEXTO DE QUIC

Actualmente, cuando navegamos por diferentes sitios web hacemos uso del protocolo TCP, ya que es utilizado por debajo de HTTP y HTTPS, siendo su puerto por defecto el 80 y el 443 respectivamente. Este protocolo nos garantiza que la información descargada llega completa y sin pérdidas. Hasta la versión de HTTP 2.0 siempre se ha utilizado TCP, ya sea empleando TLS por encima para asegurar comunicaciones PtP (*Point-to-Point*, o punto a punto) con criptografía, o directamente en el uso de HTTP donde la información se transmite sin ser cifrada.

HTTP/3 es un avance de la capa de aplicación asentado en 2021, como vemos en la cronología del protocolo representada en la Figura 9. Desde ese momento, su uso en redes ha ido en auge



Figura 9. Cronología del despliegue de HTTP. [13]

El grado de adopción de HTTP/3 los podemos ver reflejados en las siguientes estadísticas:

En 2021, tan solo el **19.5%** de los portales usan HTTP/3 mientras que un **45.5%** usaban HTTP/2 [13], como vemos en la Figura 10. Actualmente, el **29.7%** de todas los portales web con varios elementos usan HTTP/3 y un **35.3%,** HTTP/2, como representan las figuras 11 y 12. Como podemos comprobar, esto se traduce en una evolución del **10,2%** de los sitios web totales, que han pasado de HTP/2 a HTTP/3, o lo que significa lo mismo, casi un cuarto de los servicios web que en 2021 empleaban HTTP/2, a día de hoy trabajan ya con HTTP/3.

A esto, hay que sumarle la tendencia positiva que continua teniendo HTTP.3 y a la baja que posee HTTP/2 en la actualidad como vemos en las Figuras 10, 11 y 12.

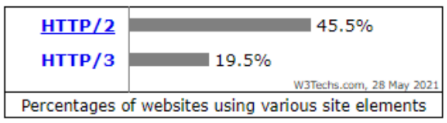


Figura 10. Despliegue de HTTP/2 y HTTP/3 en mayo de 2021. [13]

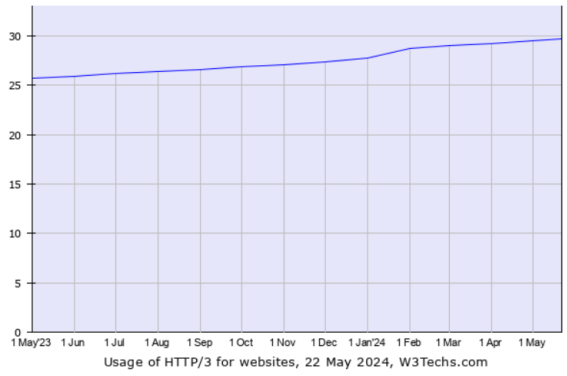


Figura 11. Uso de HTTP/3 actual [14].

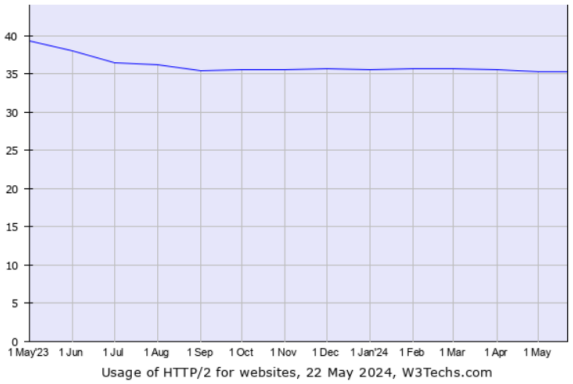


Figura 12. Uso de HTTP/2 actual [15].

El protocolo QUIC nace como alternativa a otros protocolos de transporte, cambiando el ecosistema de la capa de transporte por completo, adelantando a TCP y UDP debido a una mayor rapidez y seguridad respectivamente. El estándar HTTP 3.0 basa su capa de transporte en QUIC para lograr su objetivo de acelerar la navegación de la web, mejorar la seguridad y la eficiencia. Los detalles se muestran en la Figura 13. Además, el tráfico vaya cifrado punto a punto, siendo obligatorio el uso de HTTPS cuando solo se emplee HTTP/3.

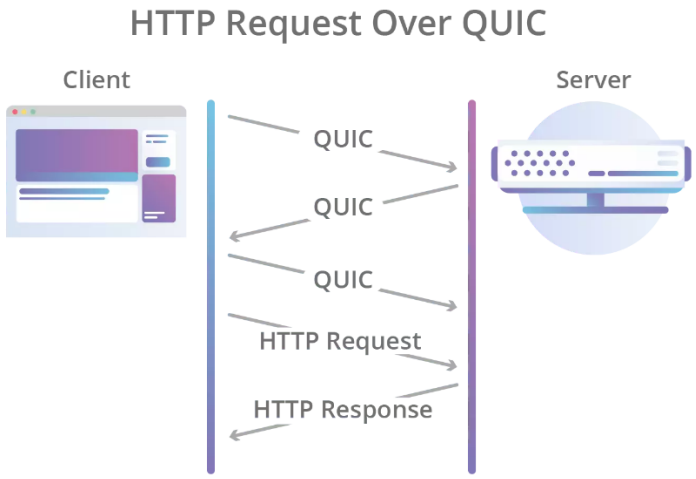


Figura 13. Empleo de QUIC en la conectividad HTTP. [17]

2.5.2. QUIC VS TCP

La principal ventaja del rendimiento de QUIC frente a TCP es su rapidez en establecer conexión, manteniendo la seguridad del enlace [18]. QUIC inicia una conexión con un único paquete (o dos paquetes si se trata de la primera vez que se establece la conexión) y transmite en ellos todos los parámetros TLS o HTTPS necesarios. En la mayoría de los casos, un cliente puede enviar datos directamente a un servidor sin que este tenga que enviar una respuesta, mientras que TCP debe obtener y procesar la confirmación del servidor. Además de esto, otras mejoras que ofrece son:

Permite conexiones multiplexadas**,** al contrario que TCP que recurre a puertos TCP fijos, haciendo imposible que un cliente se comunique con el servidor a través de varios puertos en una única conexión. QUIC soluciona esto recurriendo a una identificación de la conexión de 64 bits y a diferentes "*streams*" para transportar los datos en una conexión, que no están vinculadas a un puerto especifico (en este caso el UDP), a una IP o a un punto final determinado, siendo así posible la modificación de los puertos y de las direcciones IP.

Adjudicación de números de secuencia únicos**,** independientemente de que se trate de un segmento original o de uno reenviado. TCP, al contrario, no realiza esta distinción, haciendo imposible determinar el estado de una secuencia de paquetes. Esto permite una estimación más precisa del RTT (*Round Trip Time*).

Corrección de errores hacia delante**,** emplea paquetes FEC (*forward error correction* o corrección de errores hacia adelante) para solucionar las pérdidas de paquetes y poder reconstruirlos [19].

2.5.3. FORMATO DE LOS PAQUETES QUIC

Los paquetes QUIC pueden dividirse entre paquetes normales y paquetes especiales. Todos estos paquetes deben ser dimensionados para encajar en una MTU (unidad máxima de transferencia) para evitar fragmentación IP: 1370 bytes/paquete en caso de IPv4 y 1350 en el de IPv6. Los paquetes QUIC constan de la carga de datos útil y una cabecera de entre 2 y 19 Bytes con el siguiente formato:

- *Flags* Públicos [8 bits]. Informa si la cabecera contiene la versión del protocolo, si es paquete *reset* o no, el tamaño del ConnectionID y el número de bytes de orden inferior presentes en cada paquete

- ConnectionID [64 bits]. Número aleatorio generado por el cliente que identifica la conexión entre cliente y servidor. Este campo es el responsable de que QUIC pueda sobrevivir a cambios de dirección IP (al contrario que TCP) y de traducciones NAT (*Network Address Translation*) ya que no se modifica durante estas migraciones

- Versión de QUIC [32 bits]. Número indicativo que únicamente estará presente si lo indica la flag de versión

- Número de paquete [X bits]. Número de paquete de entre 8 y 48 bits que depende del valor del flag correspondiente.

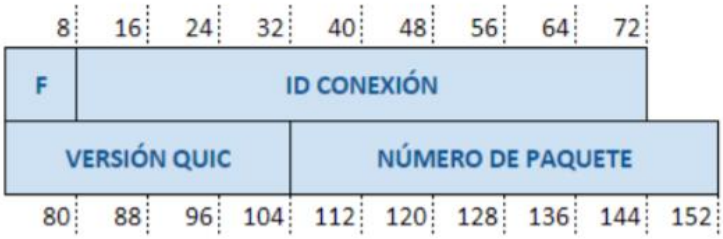


Figura 14. Cabecera de un paquete en QUIC. [13]

Los paquetes especiales son los paquetes de negociación de versión, enviados por el servidor para especificar las versiones del protocolo soportadas; y los paquetes de reset públicos, que contienen los *flags* públicos, un ConnectionID y el resto de paquete se halla codificado con los datos de establecimiento de conexión. [20]

Los paquetes normales o estándar están autenticados y encriptados (excepto la cabecera pública), lo que deniega la capacidad de inyección o de manipulación de datos por obra de terceros. Tras desencriptar el contenido del paquete, el texto plano comienza con una cabecera privada, donde se encuentra la información de FEC, seguido del tipo de mensaje y su carga útil.

2.5.4. BALANCEO DE CARGA E IMPLEMENTACIÓN CON QUIC

Actualmente, en la mayoría de los centros de datos (CPD) se encuentra que el tráfico predominante se produce entre clientes y servidores, denominado tráfico Norte-Sur, a diferencia de la situación que se producía hace unos años donde el tráfico predominante era el tráfico Este-Oeste [21] (tráfico de datos entre servidores). Este escenario conlleva a situaciones en las que dichos servidores no son capaces de entregar todo el flujo que se les requiere debido a la alta demanda de los clientes.

Surge así como solución, la inclusión de entornos *cloud* y de virtualización, logrando con ello desgranar las aplicaciones en distintos módulos distribuidos por todo el centro de datos. Con la aparición de estas nuevas tecnologías se aumentaría la conectividad entre servidores, (tráfico Este-Oeste) facilitando el correcto funcionamiento de todos los servicios y asegurando la trasmisión de información aun en situaciones de mayor volumen de tráfico.

Otra solución, no excluyente de la anterior, es la implementación de un balanceador de carga. La misión de estos, como indica su nombre, es repartir la carga entre varios recursos, aportándole un peso indicado a cada recurso para asegurar la calidad de tráfico. Es decir, balanceando el tráfico entre varios servidores que proporcionen los mismos servicios, se puede conseguir que la disponibilidad de ese recurso aumente de manera que siempre pueda ser accesible. [22]

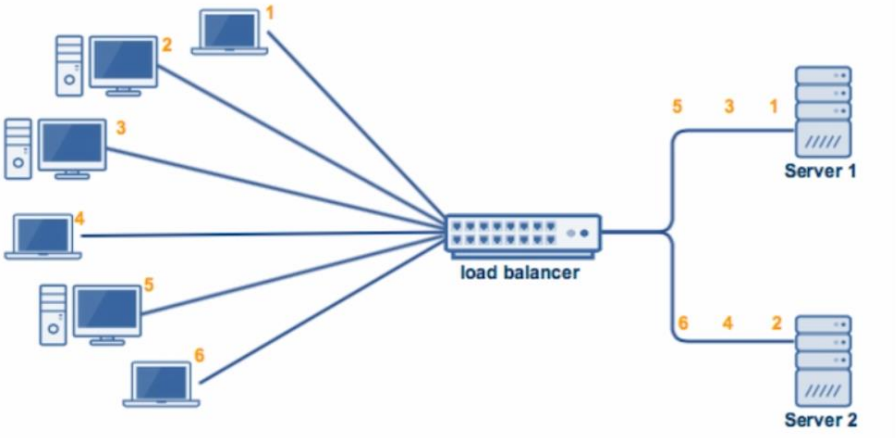


Figura 15. Red con balanceador de carga *Round Robin*. [22]

Entre todos los métodos usados para balancear el tráfico entre servidores, los siguientes son los más habituales [23] :

- ***Round Robin***: Reparte la carga de manera circular ofreciendo exactamente el mismo tráfico a cada uno de ellos. Este método es el más adecuado si los paquetes enviados tienen el mismo tamaño y no se le quiere dar prioridad a ninguno de ellos. Este se ve en la Figura 15.

- ***Round Robin* con pesos**: Este es similar al anterior solo que a cada recurso se le asigna un peso distinto, pudiendo así configurar prioridades. Esta manera se utiliza más cuando se tratan con

paquetes de distintos tamaños. El punto negativo de este tipo de balanceo es que tiene un comportamiento fijo, no teniendo en cuenta el estado de la red en cada momento.

- **Menos conexiones:** Configurándolo de este modo el tráfico se dirige hacia el servidor con menor número de conexiones activas en ese momento, intentando así que todos tengan la misma carga. Pero nada más lejos de la realidad, ya que no se tiene en cuenta la capacidad de cada una de las conexiones.

- **IP-Hash:** Esta guarda las direcciones IP de cada una de las peticiones y siempre manda su tráfico hacía el mismo terminal. Para conseguir esta persistencia, la configuración requiere del uso de memoria.

Un problema a tener en cuenta es que los conmutadores actuales colocados en los CPD no son programables, para poder implementar esta tecnología habría que invertir una gran cantidad de dinero para esta modificación. Es por esto por lo que adquiere tanta importancia el cambio del paradigma actual hacia el empleo de SDN, conmutadores programables, y redes versátiles y eficientes.

Como hemos indicado anteriormente, en la cabecera de los paquetes QUIC se encuentra el ConnectionID. Este campo adquirirá una importancia vital en las conexiones nuevas ya que hace que sean robustas en caso de reasignación de NAT (*Network Address Translation*) y de cambio de IP y será empleado por los extremos de las conexiones para dirigir los paquetes a su par.

QUIC permite a los servidores o balanceadores de carga codificar información de enrutamiento útil en el ConnectionID. Por ejemplo, para clientes con IPs dinámicas o puertos de conexión cambiantes, QUIC fomenta que los servidores proporcionen varios ConnectionIDs en paquetes protegidos para facilitar las conexiones con el servidor mientras se mueven por el mundo. Para estas acciones, el balanceador de carga tiene una función primordial debido a que es el elemento de la red que posee el contexto de la información de enrutamiento útil en sus tablas de reenvío.

El uso de QUIC genera un cambio en la categorización de los paquetes en las tablas internas del balanceador. Un balanceador de carga estándar enruta los paquetes según su protocolo IP y número de puerto, asegurando este un valor estable en el tiempo de vida de la conexión. Sin embargo, para una conexión QUIC, los balanceadores de carga (QUIC-LB) solo es necesario el uso del campo ConnectionID para el enrutamiento de los paquetes.

Para lograr la configuración de estos QUIC-LB, se necesita construir una tabla que guarde los ConnectionID de los paquetes que circulen a través del dispositivo y relacionarlos con su servidor de origen. Esto genera una protección ante cambios de IP y reasignaciones de NAT, además de una seguridad asegurada debido a que todos los datos se transmiten cifrados. No obstante, mantener esta tabla puede ser costoso en caso de que no se compartan los estados entre el balanceador de carga y los servidores.

Para liberar esa tabla, una solución propuesta es que se emplee un algoritmo común entre balanceador de carga y servidor para codificar el mapeo del servidor en un ConnectionID que contenga parámetros compartidos entre ambos dispositivos. Este mapeo solo sería detectable por observadores que tengan consciencia de los parámetros, preservando la menor vinculación posible entre dispositivos.

2.6. TECNOLOGÍA Y REDES 5G

Los primeros desarrollos en redes 5G surgen a finales de 2018, estandarizándose en 2019 por primera vez en la *Release* 15 de 3GPP [24] (*3rd Generation Partnership Program*) y manteniéndose tras eso en las posteriores *Releases*. El objetivo principal de esta quinta generación de redes móviles es distinto al de las anteriores generaciones como 3G o 4G, cuyo objetivo pasaba principalmente por ofrecer una mejoría notoria en sus prestaciones al tráfico de voz y de datos. Tal como lo muestra la Figura 16, el estándar 5G se construye de cara a tres finalidades principales de las comunicaciones, pensados para varios casos de uso definidos por la ITU (Unión Internacional de Telecomunicaciones) en 2020 [25]:

eMBB (*enhanced Multi BroadBand*), o **banda ancha móvil mejorada.** Se debe al crecimiento de ancho de banda requerido por las personas relativo a contenidos multimedia, paquetes de datos y otros servicios, con velocidades superiores a 100Mbps para la conexión de usuario. Además, busca remediar los problemas que pueda generar el aumento en la densidad de tráfico debido al número creciente de usuarios en red y de hacer que este aumento sea imperceptible en la experiencia de usuario, superando capacidades de tráfico de 10 Mbps/m^2.

uRLLC (*ultra-Reliable and Low Latency Communications*), o **comunicaciones de máxima fiabilidad y baja latencia.** Se trata del servicio a comunicaciones de máxima prioridad (como la conducción automática asistida, las consultas quirúrgicas a distancia o la realidad virtual) alcanzando latencias inferiores a 1ms y soportando la movilidad de dispositivos a velocidades superiores a 500 km/h.

MMTC (*Massive Machine Type Communications*), o **comunicaciones tipo maquina masivas.** Se caracteriza por ser un tipo de comunicaciones que constan de un número de dispositivos conectados muy elevado que, por lo general, trasmiten un volumen relativamente bajo de datos no sensibles al retardo. Los dispositivos deben ser de bajo coste y con una prolongada duración de la batería. Está pensado para el despliegue de forma masiva de dispositivos IoT, permitiendo densidades de conexión superiores al millón de dispositivos por metro cuadrado.

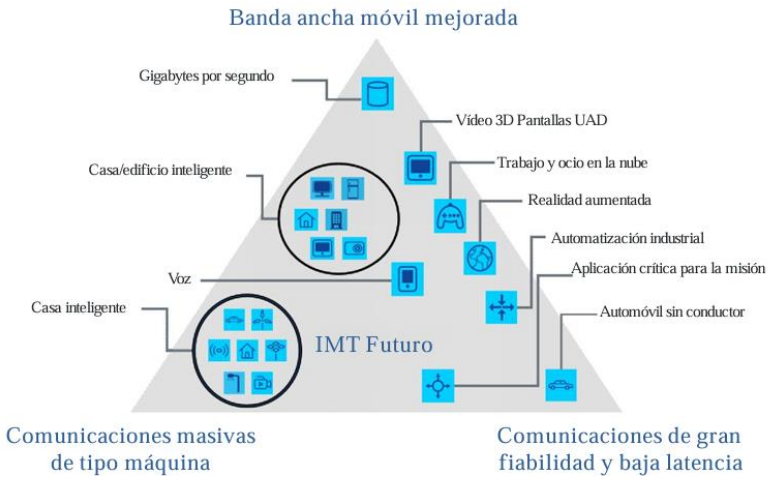


Figura 16. Casos de uso de las comunicaciones con 5G. [25]

2.6.1. ARQUITECTURAS

La tecnología 5G se estandariza por primera vez en la *Release* 15 de 3GPP con el despliegue de la versión NSA (*Non-Stand Alone*, no autónomo). Esta versión está diseñada como un paso intermedio entre la tecnología móvil anterior, 4G LTE (*Long Term Evolution*) y lo que posteriormente se estandarizaría en la *Release* 17 [26] como 5G SA (*Stand Alone*, autónomo) [27]. La evolución de las redes, que se ve referenciada en Figura 17, es provocada por la estandarización de elementos de red programables (*Future Router* en la figura).

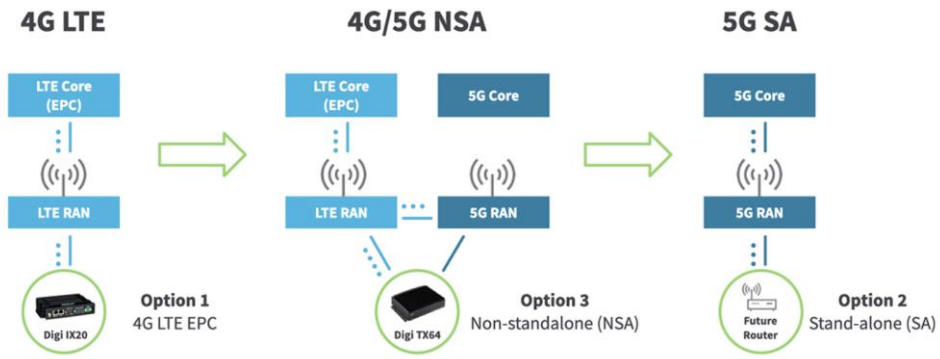


Figura 17. Evolución de las redes. Operadora DIGI [28]

5G-NSA

Los despliegues 5G-NSA se caracterizan por el uso del núcleo de red EPC (*Evolved Packet Core*) nativo de 4G LTE, en vez del suyo propio, para el plano de señalización y control. Estas arquitecturas permiten la implantación de algunas novedades como NR (*New Radio*), la tecnología de acceso radio característica de 5G, sin embargo no son capaces de ofrecer todas las ventajas asociadas a 5G en su totalidad.

5G-SA

Los escenarios de 5G SA actualmente están aún en proceso de despliegue. La principal diferencia que incluyen es la utilización del núcleo de red 5G para el plano de señalización y control, operando de manera independiente a la red 4G LTE. Esto le permite alcanzar las capacidades de 5G de manera completa gracias a tecnologías clave como la segmentación de red (*network slicing*), la computación frontera (*edge computing*) y la arquitectura basada en servicios (SBA, *Service Based Architecture*) entre otras. [29]

La cualidad principal en la arquitectura del propio núcleo de red es la separación de los planos de control y usuario, hasta ahora procesados conjuntamente, como hemos visto previamente en la inclusión de las SDN y la fragmentación del plano de control en servicios, denominados funciones de red (FN, *Network Function*) comunicados entre si mediante peticiones HTTP. [30]

El 5G se diseñó desde cero y las funciones de la red se dividen por servicios. Por eso, esta arquitectura también se llama núcleo 5G Arquitectura basada en servicios (SBA o *Service-based Architecture*). [31] Esta arquitectura, representada en la Figura 18, es más compleja que las arquitecturas de 3G y 4G, sin embargo, es necesaria para asegurar el servicio de todos los tipos de servicios y casos de uso que ofrece la tecnología.

Su funcionamiento, sin entrar en demasiado detalle, consiste en que los equipos de usuario (5G-UE) se conectan al núcleo 5G a través de la red de acceso radioeléctrico y a las redes de datos (DN), como Internet, de forma simultánea e independiente, debido a la liberación de los planos funcionales de la red. Para el correcto funcionamiento de las redes, tienen un papel principal las funciones de red, como hemos comentado previamente:

La función de acceso y movilidad (AMF) actúa como punto de entrada único para la conexión del equipo de usuario. Basándose en el servicio solicitado por el UE, la AMF selecciona la respectiva función de gestión de sesión (SMF) para gestionar la sesión de usuario.

La función de plano de usuario (UPF) transporta el tráfico de datos IP (el plano de usuario) entre el equipo de usuario (UE) y las redes externas.

La función de servidor de autenticación (AUSF) permite a la AMF autenticar al UE y acceder a los servicios del núcleo 5G.

Otras funciones como función de control de políticas (PCF), la función de aplicación (AF) y la función de gestión unificada de datos (UDM) proporcionan el marco de control de políticas, aplicando las decisiones de política y accediendo a la información de suscripción, para gobernar el comportamiento de la red. [32]

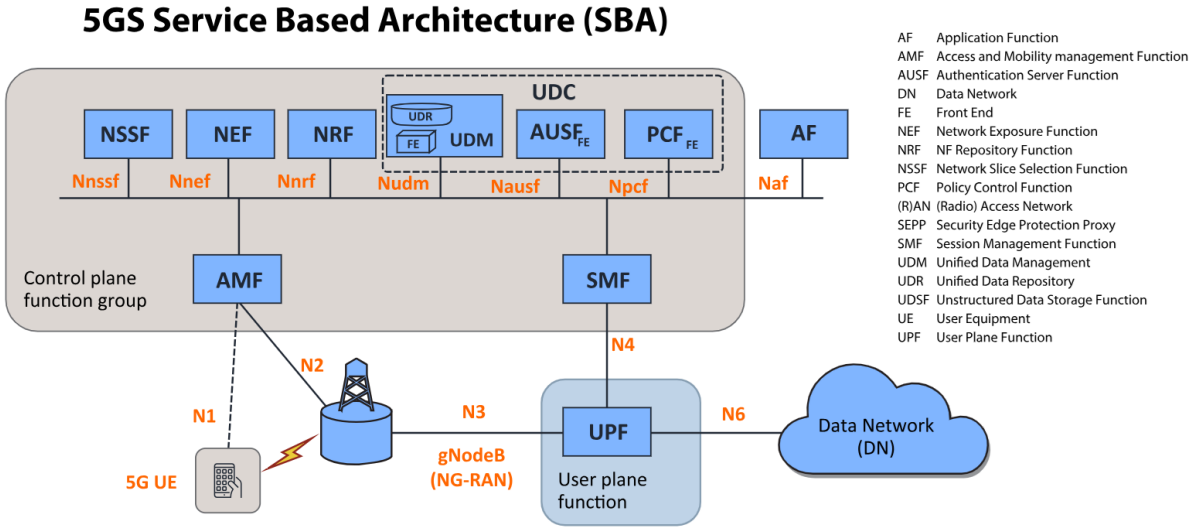


Figura 18. Diagrama de SBA [33]

2.6.2. APLICACIONES DE PLANOS DE DATOS PROGRAMABLES A 5G

La programabilidad del plano de datos ofrece nuevas funcionalidades que implementadas a arquitecturas de 5G, suponen un avance significativo en el alcance de sus objetivos tecnológicos. Numerosos artículos han explorado el potencial de estas funciones [33]. Los principales casos de uso reflejados en la literatura actual son:

Tunelación y reenvío de paquetes. Al reasignar estas funciones a dispositivos de plano programables, se muestra una reducción de la latencia en las comunicaciones muy significativa en comparación con las implementaciones software empleadas anteriormente.

Segmentación de Red. La segmentación se realiza directamente desde dispositivos de plano programables en distintas secciones de la arquitectura 5G. Estos dispositivos son capaces de segmentar distintos tipos de tráfico con menor latencia y menor porcentaje de pérdida de paquetes.

Ciberseguridad. Se realiza detección de ataques DDoS (*Distributed Denial-of-Service*, denegación de servicio distribuido) con la ayuda de *switches* programables para extraer información de los paquetes en lugar de las soluciones basadas en el CPU previas. También se implementan cortafuegos para bloqueo de tráfico malicioso.

Telemetría en banda. Se recolecta información de la red directamente desde el plano de datos, lo que implica una reducción de la latencia significativa y baja pérdida de paquetes .

Descarga del plano de control. Almacenar datos de estados en el hardware del *switch* aumenta el rendimiento y disminuye la latencia de manera significativa. En más detalle, descargar un porcentaje de los mensajes de control a dispositivos programables resulta en un aumento notable del rendimiento en comparación con implementaciones tradicionales que no implementan estas descargas de datos.

3. DESARROLLO Y RESULTADOS

3.1. INSTALACIÓN DE HERRAMIENTAS PARA AMBIENTE P4

La mayoría de las implementaciones relacionadas con P4 requieren herramientas similares. Por ello, hemos decidido unificar el uso estas herramientas y dependencias para ambos escenarios. Esta decisión se debe a la escasez de referencias sobre implementaciones que usen el protocolo QUIC y la mayor variedad de opciones que ofrece TCP, por lo que buscaremos una implementación de TCP acorde a las herramientas que requiera la implementación en QUIC. La principal herramienta instalada es la ETH-P4 VM, una máquina virtual instalada en este caso en Oracle VM VirtualBox (también aceptada por QEMU), que se ofrece preconfigurada para su uso con archivos P4. Para su instalación tan solo es necesario copiar el repositorio de GitHub donde se ofrece y ejecutar un comando de inicialización de VirtualBox, como podemos comprobar en el siguiente extracto:

```
& git clone https://github.com/nsg-ethz/p4-utils
& cd p4-utils/vm
& ./build-virtualbox.sh [--cpus 4] [--disk_size 25000] [--memory 4000] [--vm_name p4] [--username p4] [--password p4]
```

Una vez instalada esta máquina virtual, emplearemos el usuario P4 autoconfigurado que presenta el entorno, para poder instalar todas las dependencias y las carpetas de archivos de los escenarios que implementaremos. La mayoría de las instalaciones de estas dependencias podremos unificarlas, facilitando de esta manera el proceso, con la instalación de p4-utils a través de Mininet. Este último, es un emulador utilizado para el despliegue de redes sobre los limitados recursos que poseemos en nuestra máquina virtual, que emplearemos en la construcción de ambos escenarios.

p4-utils, es una extensión de Mininet que facilita la construcción y entendimiento de escenarios P4. Este repositorio ofrece un archivo "*bash*" que instala todas las dependencias que se requiere: *PI Library Repository*, *Behavioral model* (bmv2), p4c, Mininet y FRRouting.

```
& git clone https://github.com/nsg-ethz/p4-utils.git
& cd p4-utils
& sudo ./install.sh
& curl -sSL https://raw.githubusercontent.com/nsg-ethz/p4-utils/master/install-tools/install-p4-dev.sh | bash
```

Con esto, ya contaremos con todas las dependencias necesarias para la implementación del primer escenario, es decir, el uso de TCP. Para la implementación con QUIC, será necesario la descarga de dos carpetas adicionales: picoquic y picotls. A su vez, estas requieren de la instalación de OpenSSL, una biblioteca de criptografía que ofrece una aplicación de código abierto del protocolo TLS.

Para la instalación de estas dependencias las copiaremos de sus respectivos librerías en GitHub en orden: primero https://github.com/h2o/picotls y tras ello, https://github.com/private-octopus/picoquic .

La instalación de OpenSSL conlleva una mayor complejidad ya que las librerías se hallan desactualizadas y requieren una versión que ya no está disponible, lo que nos requerirá navegar en las librerías de la biblioteca, resultando la instalación más laboriosa.

3.2. PRIMER ESCENARIO, PRUEBA DE TCP

3.2.1. EXPLICACION DE LAS PRUEBAS A REALIZAR

Para el estudio del protocolo TCP empleado, se ha optado por un escenario que emplea balanceo de cargas por el método ECMP (*Equal Cost Multiple Pathing*, caminos múltiples de igual coste) para comprender de manera completa el papel que adquiere el protocolo TCP en el archivo .p4 que controla los paquetes de la red. Este escenario se ha basado en un ejercicio propuesto en la rama del repositorio p4-learning [35], el cual incluye una compilación de documentos, ejercicios, ejemplos y transparencias del lenguaje P4.

La función del protocolo TCP, como veremos en los archivos .p4 que controlan la red (ecmp.p4), es la de encaminar correctamente los paquetes, dirigiendo la retransmisión de estos en las tablas de los *switches* (ecmp_group_to_nhop y ipv4_lpm), que tienen en cuenta las direcciones IP, y los puertos TCP de origen y destino de los paquetes que ofrece TCP. Estos atributos son la base de varias funcionalidades esenciales para el procesamiento y encaminamiento de paquetes de red:

- Acción de tablas y las reglas de enrutamiento: La tabla ipv4_lpm del controlador general depende directamente de la dirección IP de destino de los paquetes y actúa en la decisión del próximo salto, set_nhop y en la acción ecmp_group que a su vez, toma los puertos TCP para mejorar la decisión de su enrutamiento. La tabla ecmp_group_to_nhop tiene en cuenta el ecmp_group_id y el ecmp_hash para decidir el siguiente salto de un paquete o descartarlo.
- Función *hash* para el balanceo de carga: La acción ecmp_group realiza un cálculo crucial para el balanceo de carga a través de ECMP. Al incluir los puertos TCP, el algoritmo de *hash* puede distribuir equitativamente el tráfico entre múltiples rutas de igual costo, mejorando la eficiencia de la red.
- Decremento de TTL; La acción set_nhop actualiza las cabeceras de los paquetes, modificando sus atributos de origen, destino y TTL con cada salto de elemento en la red.

En resumen, TCP aporta unos atributos que son esenciales en la totalidad de las comunicaciones que emplean este protocolo, entre ellas, en el balanceo de carga estudiado. La inclusión de estos campos TCP en las acciones y tablas permite la gestión del tráfico y eficiencia en la red.

Además de esto, consideraremos implementar ficheros de configuración CLI de los *switches*, a los que llamaremos sX-command.txt, en los que describiremos cómo queremos que actúe cada *switch*, configurando sus tablas de reenvío con la información obtenida previamente por los protocolos y almacenadas en las tablas ipv4_lpm y ecmp_group_to_nhop

La red elegida para esta primera prueba consiste en un escenario con dos *hosts*, cada uno conectado a un *switch* principal. Entre estos *switches* principales, se conectarán varios *switches* secundarios en paralelo que dividirán el tráfico en tantos caminos como *switches* se utilicen. En este caso, como muestra la Figura 19, implementaremos la red con dos *hosts* conectados directamente por cuatro caminos de igual coste, es decir, emplearemos cuatro *switches* secundarios conectados a los dos principales, que son los que se conectarán directamente a los *hosts*. Con esta topología, los paquetes deberían elegir un camino de manera aleatoria (■, ■, ■ o ■) con una probabilidad del 25% cada vez, teniendo en cuenta los parámetros proporcionados por el protocolo.

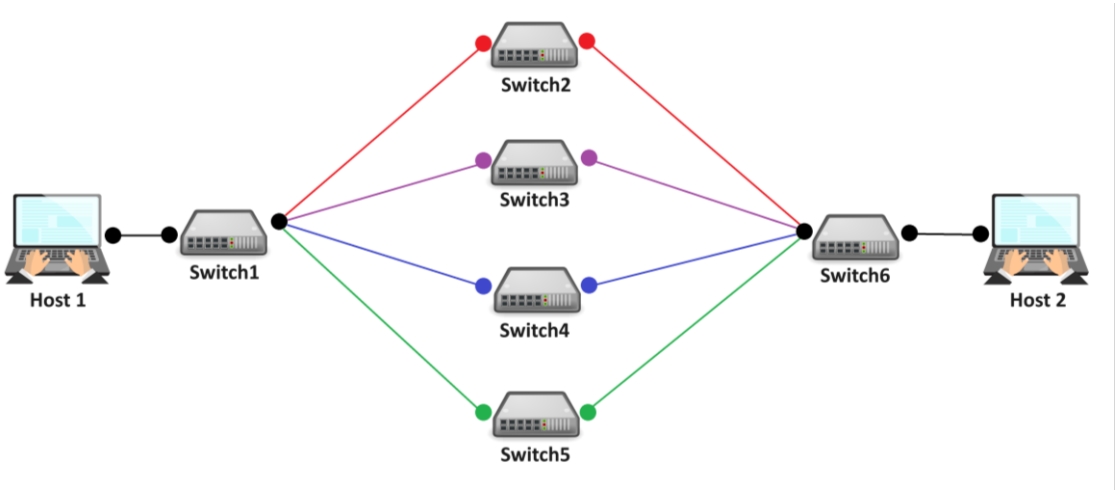


Figura 19. Topología del escenario 1.

La carpeta de archivos necesaria para esta implementación tendrá la forma mostrada en la Figura 20. La definición de esta topología se realiza en los archivos p4app.json y network.py. La carpeta p4src contiene los archivos P4 principales para el funcionamiento de la red. La carpeta sw-commands contiene los archivos .txt necesarios para configurar las tablas de cada *switch* en la red para el correcto reenvío de los paquetes. Por último, el archivo send.py es un archivo Python externo que emplearemos, el cual se encargará de enviar tantos paquetes como queramos (en este caso 1000) para hacer una prueba global de la red.

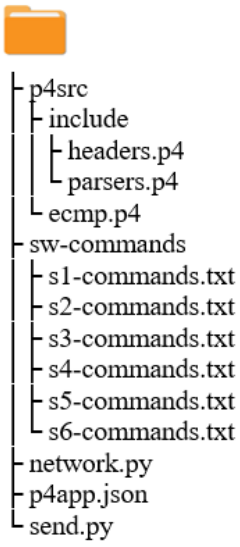


Figura 20. Representación de la carpeta con los archivos necesarios para el escenario 1

Todos estos archivos completos son anexados en el Anexo A.

Los ficheros p4app.json y network.py definen la topología de la red que usaremos mediante llamadas a los métodos "net.addElemento" dentro del fichero network.py, coherente con las definiciones de la red del archivo JSON. Algunas de las llamadas coherentes más significativas de ambos archivos son la asignación de la estrategia de la topología, la inclusión de hosts, *switches* y links entre elementos, por ejemplo :

```
//NETWORK.PY
    net.addP4Switch('s1', cli_input='sw-commands/s1-commands.txt')
    net.addHost('h1')
    net.addLink("h1", "s1")
    net.mixed()


//P4APP.JSON
    "topology": {
      "assignment_strategy": "mixed",
      "links": [["h1", "s1"], […]],
      "hosts": {"h1": {}, {…}},
      "switches": {"s1": {"cli_input": "sw-commands/s1-commands.txt"}, {…}}
```

Los ficheros sw-commands.txt son ficheros que definen las acciones de los *switches* CLI para el encaminamiento de paquetes mediante el llenado de sus tablas. Los *switches* principales constan de una configuración parecida, al ser una topología simétrica y los cuatro *switches* intermedios poseen la misma configuración exacta, al tratarse de caminos de igual coste, origen y destino.

Como podemos comprobar, la configuración de los dos *switches* principales es idéntica, tan solo percibimos cambios en las direcciones de origen y destino para los puertos simétricos. Por otro lado, comprobaremos que solo hemos configurado un ecmp_id (=1), ya que en este caso solo es necesario emplear. La función que realiza ecmp_id es que permite encaminar paquetes desde una dirección IP hasta unos puertos concretos.

En este caso preciso tan solo necesitamos un ecmp_id para los cuatro puertos, pero es útil su configuración para otros casos con varias direcciones IP o con puertos específicos para IPs determinadas, por ejemplo, usando los dos puertos superiores para IP1, y los dos inferiores para IP2.

Para la configuración P4 de la gestión de los paquetes en la red, contamos con el fichero ecmp.p4, mencionado en los ficheros de la topología como el código que instruye a la red completa. Este código, como cualquier código P4 consta de un parseador, un deparseador (definidos en un fichero parsers.p4, para una mayor limpieza en el código) y de unas instrucciones en el pipeline de los datos entre las que constituyen principalmente las entradas *Ingress* y *Egress*, para construir virtualmente los *switches*.

PARSER (Código anexado, parser.p4). Se define una máquina de estados finita en la que cada estado extrae sus cabeceras propias, las guarda en variables definidas (las cabeceras necesarias también son definidas en otro fichero headers.p4) y pasa al siguiente estado dependiendo del valor del encabezado que indica el tipo de protocolo que usa. Podemos visualizarlo en la Figura 21.

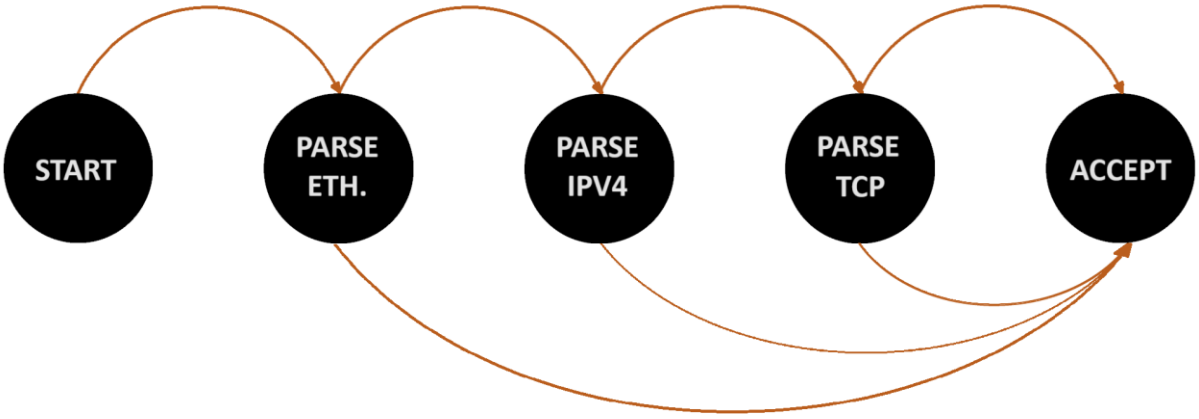


Figura 21. Representación de la máquina de estados del *Parser* TCP

Start. Estado inicial, cambia directamente hacia el estado de Ethernet.

Parse Ethernet. Extrae y almacena la cabecera Ethernet (hdr.ethernet) del paquete que contiene campos como srcAddr, destAddr y etherType. Si el campo etherType (hdr.ethernet.etherType) es de tipo IPv4 cambia al estado de IPv4, si contiene cualquier otro valor, completa el procesamiento del paquete cambiando hacia el estado final.

Parse IPv4. Extrae y almacena la cabecera IPv4 del paquete en la estructura hdr.ipv4. Si el campo protocol (hdr.ipv4.protocol) de esa cabecera es "6" es decir, que emplee el protocolo TCP, cambia al estado de TCP, si contiene cualquier otro valor, completa el procesamiento del paquete cambiando hacia el estado final.

Parse TCP. Extrae la cabecera TCP y la almacena en la estructura hdr.tcp. Tras esto cambia al estado final

Accept. Estado final, finaliza la función del *Parser* concluyendo el procesamiento del paquete.

DEPARSER (Código anexado, parser.p4). Se limita a agregar las cabeceras parseadas de nuevo a los paquetes procesados para su salida del *switch*. Solo serán emitidas si fueron válidos en el *Parser*, es decir, si cumplieron los requisitos para pasar por todos los estados de la máquina.

PIPELINE (Código anexado, ecmp.p4). Consta de todos los demás módulos que constituyen el *switch*.

- **VerifyCheckSum.** En nuestro caso no es necesario realizar ninguna verificación con las cabeceras ni con los metadatos para la integridad de los datos.
- **Ingress.** Esta sección es la más importante del programa, siendo así también la más extensa. Instruye las acciones a realizar por el programa:
    - drop. Marca el paquete para ser descartado, utilizando la función mark_to_drop que modifica el campo standard_metadata.drop
    - ecmp_group. Realiza un cálculo *hash* sobre varios campos del encabezado IPv4 y TCP para determinar el balanceo de carga ECMP. El resultado del *hash* se almacena en meta.ecmp_hash y el grupo ECMP se almacena en meta.ecmp_group_id
    - set_nhop. Configura el siguiente salto para el paquete. Cambia la dirección MAC de origen, configura la dirección MAC de destino al parámetro dstAddr proporcionado, establece el puerto de salida y decrementa el TTL (Time to Live) del encabezado IPv4.

Define las tablas que serán necesarias para el encaminamiento de los paquetes en los *routers*:

- ecmp_group_to_nhop: Se utiliza para determinar el siguiente salto basado en el grupo ECMP y el *hash* calculado relacionando estos parámetros a una acción específica (set_nhop o drop).
- ipv4_lpm: Determina que acción debe ejecutarse, realizando una búsqueda de prefijo más largo (LPM - Longest Prefix Match) basada en la dirección de destino IPv4. Si no hay coincidencia se descartará el paquete por defecto.

Aplica las tablas (en el bloque *apply*) se verifica si el encabezado IPv4 es válido (hdr.ipv4) y se aplica la tabla ipv4_lpm. Si la acción ejecutada es ecmp_group, entonces se aplica la tabla ecmp_group_to_nhop para determinar el siguiente salto.

- **Egress.** El procesamiento de *egress* se realiza justo antes de que un paquete salga del *switch* y permite realizar cualquier modificación necesaria en los encabezados de los paquetes. En este caso, *egress* está vacío ya que no es necesario ningún cambio en las cabeceras de los paquetes procesados.
- **ComputeCheckSum.** Asegura que los encabezados de los paquetes sean correctos antes de su salida del Pipeline y su envío posterior. Esta labor se denomina cálculo de checksums, los cuales son esenciales para la integridad de los datos, verificando que los paquetes no han sido corrompidos durante su manipulación en el *switch*.

CONSTRUCCION DEL *SWITCH*. Se genera la arquitectura requerida para el *switch* programable (arquitectura PISA) V1Switch con los módulos descritos previamente: MyParser(), MyVerifyChecksum(), MyIngress(), MyEgress(), MyComputeChecksum(), MyDeparser().

Finalmente para generar la red emplearemos el comando sudop4run, llamando a la carpeta p4-utils instalada para que se genere de una manera más rápida el escenario de Mininet necesario para probar la implementación.

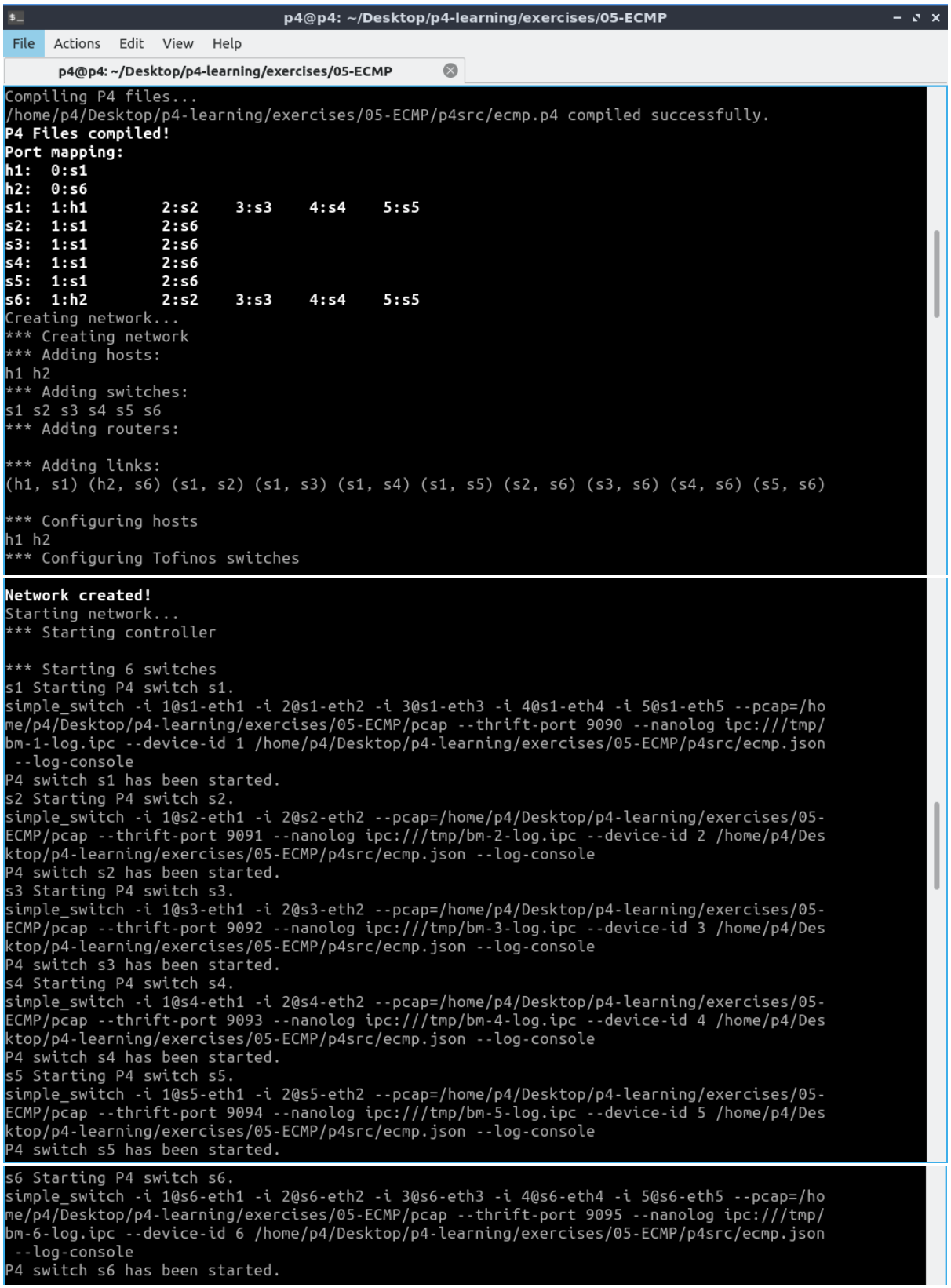
3.2.2. RESULTADOS DE RENDIMIENTO

Los resultados que pretendemos conseguir con la implementación de este escenario TCP se basan en la comprobación del correcto funcionamiento de la red: creación, conexión, balanceo de tráfico, etc. Y con ello, entender el papel que ha desempeñado TCP y los demás ficheros en la red.

Comenzaremos comprobando el código con el que responde la máquina tras realizar el comando sudo p4run para construir el escenario y abrir la ventana de Mininet necesaria. Esta respuesta se podrá ver reflejada en la Figura 22.

Al tener los *logs* de la red activados, como vemos en los ficheros de creación de la red, podemos comprobar estos ficheros que generan las trazas para comprobar el rellenado de las tablas de los *switches* en la Figura 23. Esto nos dará la idea de que las redes estarán bien configuradas en el reenvío de los paquetes.

Para comprobar la conexión podemos realizar un pingall en Mininet que se genera tras ejecutar el código general. Esto es simplemente para comprobar que llegan los paquetes, ya que la prueba total del escenario la realizaremos empleando el archivo send.py que hemos comentado antes y evaluaremos qué cantidad de los paquetes enviados con ese método son transportados por cada camino, cumpliendo así el balanceo ECMP.


p4@p4: ~/Desktop/p4-learning/exercises/05-ECMP
File Actions Edit View Help
p4@p4: ~/Desktop/p4-learning/exercises/05-ECMP
Compiling P4 files...
/home/p4/Desktop/p4-learning/exercises/05-ECMP/p4src/ecmp.p4 compiled successfully.
P4 Files compiled!
Port mapping:
h1: 0:s1
h2: 0:s6
s1: 1:h1 2:s2 3:s3 4:s4 5:s5
s2: 1:s1 2:s6
s3: 1:s1 2:s6
s4: 1:s1 2:s6
s5: 1:s1 2:s6
s6: 1:h2 2:s2 3:s3 4:s4 5:s5
Creating network...
*** Creating network
*** Adding hosts:
h1 h2
*** Adding switches:
s1 s2 s3 s4 s5 s6
*** Adding routers:
*** Adding links:
(h1, s1) (h2, s6) (s1, s2) (s1, s3) (s1, s4) (s1, s5) (s2, s6) (s3, s6) (s4, s6) (s5, s6)
*** Configuring hosts
h1 h2
*** Configuring Tofinos switches
Network created!
Starting network...
*** Starting controller
*** Starting 6 switches
s1 Starting P4 switch s1.
simple_switch -i 1@s1-eth1 -i 2@s1-eth2 -i 3@s1-eth3 -i 4@s1-eth4 -i 5@s1-eth5 --pcap=/home/p4/Desktop/p4-learning/exercises/05-ECMP/pcap --thrift-port 9090 --nanolog ipc:///tmp/bm-1-log.ipc --device-id 1 /home/p4/Desktop/p4-learning/exercises/05-ECMP/p4src/ecmp.json --log-console
P4 switch s1 has been started.
s2 Starting P4 switch s2.
simple_switch -i 1@s2-eth1 -i 2@s2-eth2 --pcap=/home/p4/Desktop/p4-learning/exercises/05-ECMP/pcap --thrift-port 9091 --nanolog ipc:///tmp/bm-2-log.ipc --device-id 2 /home/p4/Desktop/p4-learning/exercises/05-ECMP/p4src/ecmp.json --log-console
P4 switch s2 has been started.
s3 Starting P4 switch s3.
simple_switch -i 1@s3-eth1 -i 2@s3-eth2 --pcap=/home/p4/Desktop/p4-learning/exercises/05-ECMP/pcap --thrift-port 9092 --nanolog ipc:///tmp/bm-3-log.ipc --device-id 3 /home/p4/Desktop/p4-learning/exercises/05-ECMP/p4src/ecmp.json --log-console
P4 switch s3 has been started.
s4 Starting P4 switch s4.
simple_switch -i 1@s4-eth1 -i 2@s4-eth2 --pcap=/home/p4/Desktop/p4-learning/exercises/05-ECMP/pcap --thrift-port 9093 --nanolog ipc:///tmp/bm-4-log.ipc --device-id 4 /home/p4/Desktop/p4-learning/exercises/05-ECMP/p4src/ecmp.json --log-console
P4 switch s4 has been started.
s5 Starting P4 switch s5.
simple_switch -i 1@s5-eth1 -i 2@s5-eth2 --pcap=/home/p4/Desktop/p4-learning/exercises/05-ECMP/pcap --thrift-port 9094 --nanolog ipc:///tmp/bm-5-log.ipc --device-id 5 /home/p4/Desktop/p4-learning/exercises/05-ECMP/p4src/ecmp.json --log-console
P4 switch s5 has been started.
s6 Starting P4 switch s6.
simple_switch -i 1@s6-eth1 -i 2@s6-eth2 -i 3@s6-eth3 -i 4@s6-eth4 -i 5@s6-eth5 --pcap=/home/p4/Desktop/p4-learning/exercises/05-ECMP/pcap --thrift-port 9095 --nanolog ipc:///tmp/bm-6-log.ipc --device-id 6 /home/p4/Desktop/p4-learning/exercises/05-ECMP/p4src/ecmp.json --log-console
P4 switch s6 has been started.

```
*** Starting 0 routers

Network started!
Starting schedulers...
Schedulers started correctly!
Saving topology to disk...
Saving mininet topology to database: ./topology.json
Topology saved to disk!
Programming switches...
Switch s1: successfully configured with file /home/p4/Desktop/p4-learning/exercises/05-EC
MP/sw-commands/s1-commands.txt.
Switch s2: successfully configured with file /home/p4/Desktop/p4-learning/exercises/05-EC
MP/sw-commands/s2-commands.txt.
Switch s3: successfully configured with file /home/p4/Desktop/p4-learning/exercises/05-EC
MP/sw-commands/s3-commands.txt.
Switch s4: successfully configured with file /home/p4/Desktop/p4-learning/exercises/05-EC
MP/sw-commands/s4-commands.txt.
Switch s5: successfully configured with file /home/p4/Desktop/p4-learning/exercises/05-EC
MP/sw-commands/s5-commands.txt.
Switch s6: successfully configured with file /home/p4/Desktop/p4-learning/exercises/05-EC
MP/sw-commands/s6-commands.txt.
Switches programmed correctly!
Programming hosts...
Hosts programmed correctly!
Executing scripts...
All scripts executed correctly!
Distributing tasks...
All tasks distributed correctly!
s1 -> Thrift port: 9090
s2 -> Thrift port: 9091
s3 -> Thrift port: 9092
s4 -> Thrift port: 9093
s5 -> Thrift port: 9094
s6 -> Thrift port: 9095
**********
Network configuration for: h1
Default interface: h1-eth0      10.0.1.1        00:00:0a:00:01:01
**********
**********
Network configuration for: h2
Default interface: h2-eth0      10.0.6.2        00:00:0a:00:06:02
**********
Starting mininet CLI...
================================================================================
Welcome to the P4 Utils Mininet CLI!
================================================================================
Your P4 program is installed into the BMV2 software switch
and your initial configuration is loaded. You can interact
with the network using the mininet CLI below.

To inspect or change the switch configuration, connect to
its CLI from your host operating system using this command:
  simple_switch_CLI --thrift-port <switch thrift port>

To view a switch log, run this command from your host OS:
  tail -f <log_dir>/<switchname>.log
By default log directory is "./log".

To view the switch output pcap, check the pcap files in <pcap_dir>:
  for example run:  sudo tcpdump -xxx -r s1-eth1.pcap
By default pcap directory is "./pcap".

*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet>
mininet>
```


Figura 22. Generación de Mininet del escenario

Como podemos observar, se muestra por pantalla una generación correcta de la red, elemento por elemento, tanto los *switches*, como sus conexiones referenciando a los puertos del enlace, como los *hosts* y sus interfaces predeterminadas. También comprobamos que, al tener los *logs* activados, podemos ver como los archivos .txt cumplen su finalidad de configurar los *switches* con comandos CLI, como comprobamos en el archivo de trazas de creación del sw2 en la Figura 23:

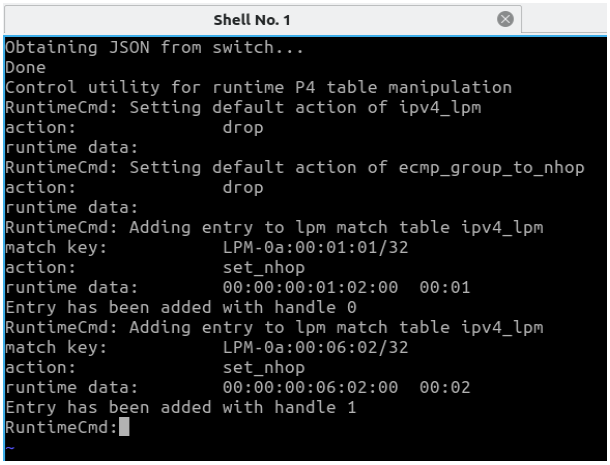


Figura 23. Agregación de entradas a las tablas de sw2

Comprobaremos el funcionamiento del programa P4 comprobando las trazas que dejan los *switches* .Para ello realizaremos dos pruebas con dos protocolos diferentes para poder ver las diferencias: realizaremos el comando h1 ping h2, visible en la Figura 24, que enviará un mensaje del protocolo ICMP (número de protocolo = 1) para comprobar la conexión. Por otro lado, usaremos netcat para enviar un archivo adicional index.txt creado en el directorio de trabajo mediante el protocolo TCP (num = 6) , de la siguiente forma, en el puerto 50 (elegido por nosotros). Los comandos realizados para este envío de paquetes TCP se ven reflejado las Figuras 25 y 26.

El envío fructífero de estos archivos lo podemos ver capturado en la Figura 27 con la herramienta Wireshark. Tras ello, estudiaremos las trazas que ha dejado del *switch* como hemos comentado previamente, adjuntadas en las Figuras 28 y 29.

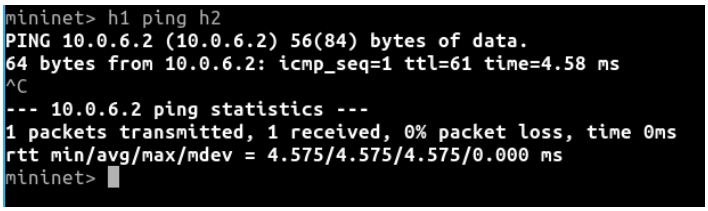


Figura 24. Comando para comprobar conexión ICMP



Figura 25. Envío de la conexión TCP

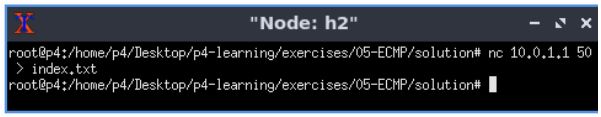


Figura 26. Recepción de la conexión TCP

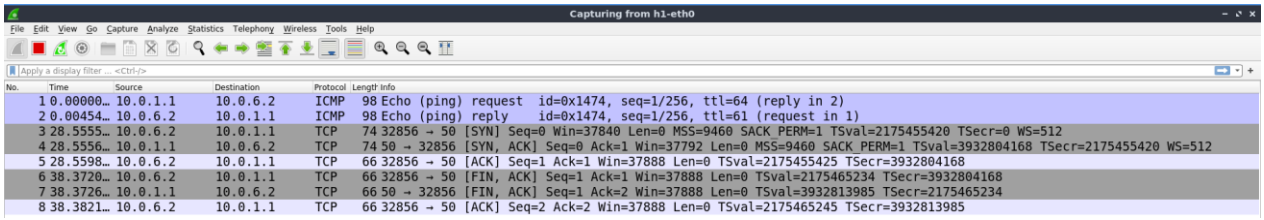


Figura 27. Visualización de los paquetes circulados en el enlace h1-s1

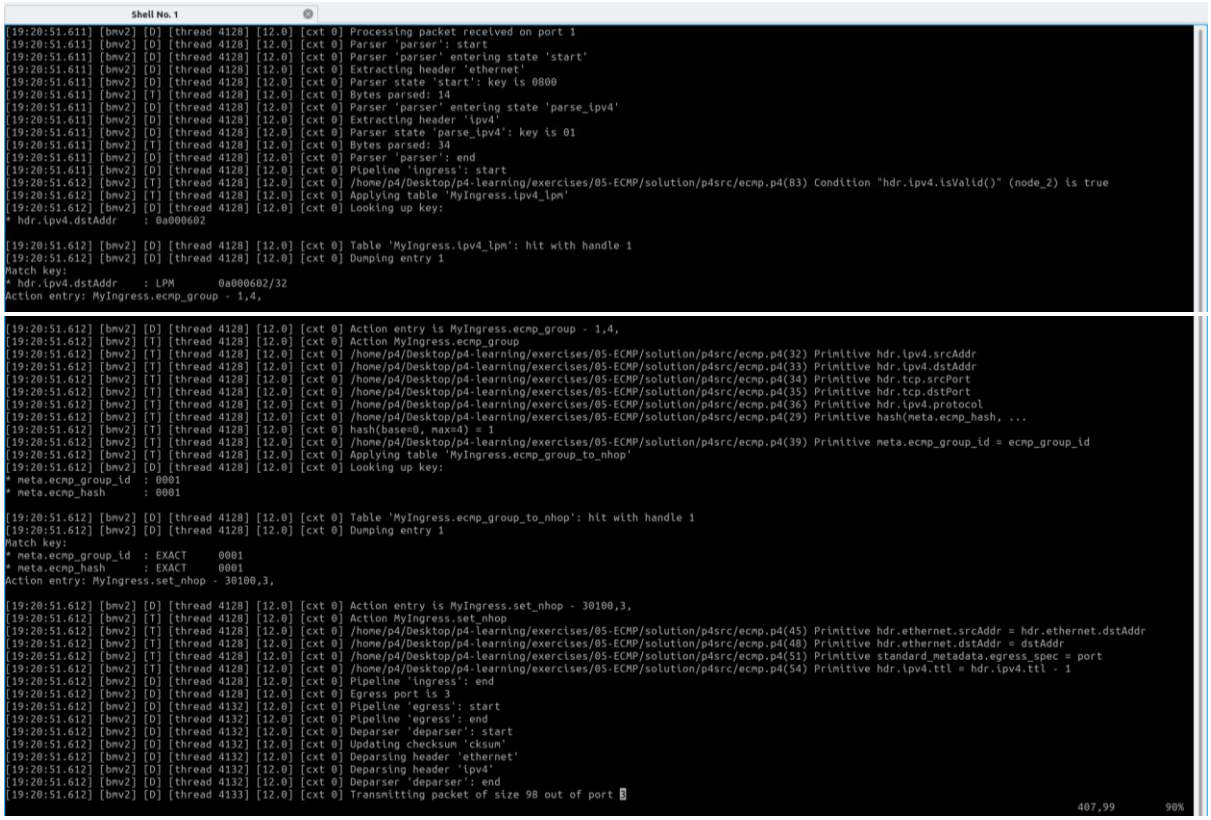


Figura 28. Trazas de la configuración P4 en sw1 tras la conexión ICMP

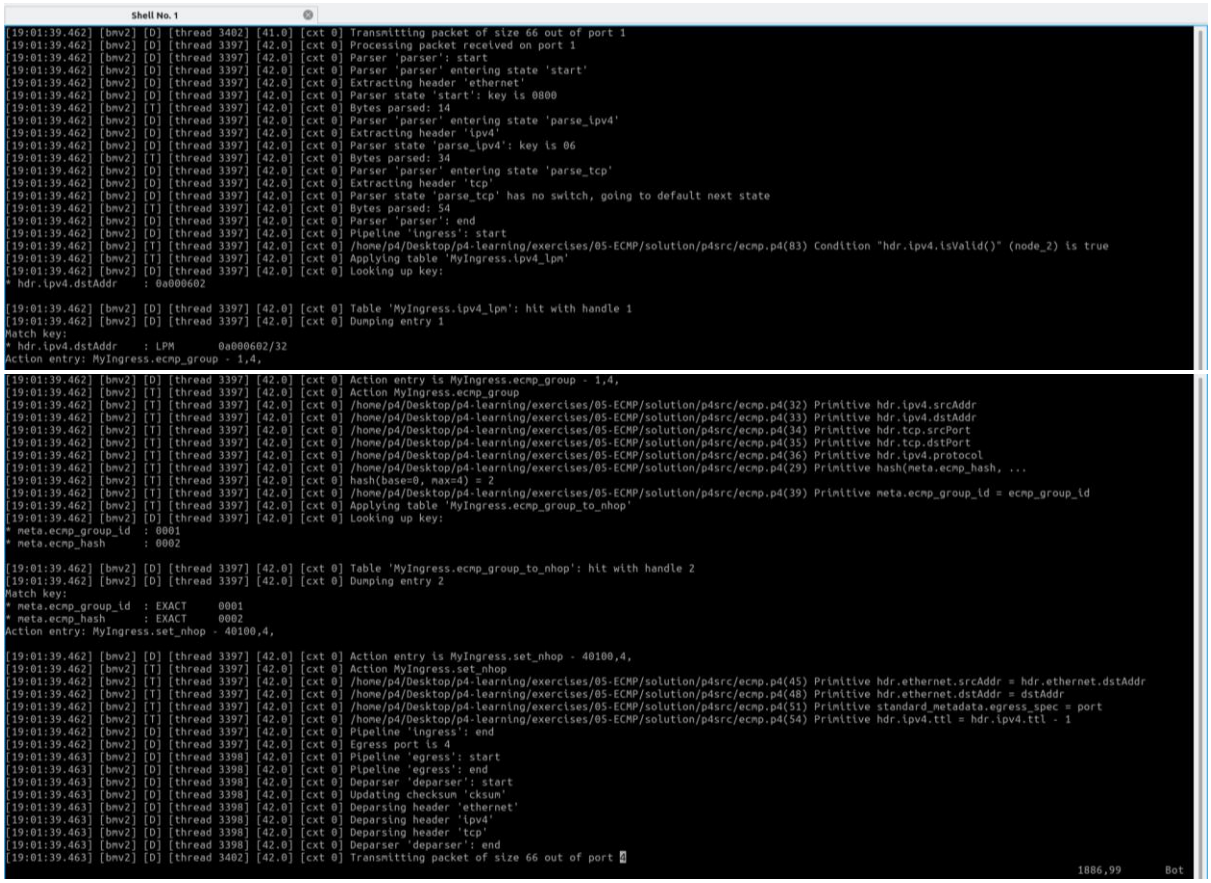


Figura 29. Trazas de la configuración P4 en sw1 tras la conexión TCP

Si analizamos las trazas que otorga el switch1, comprobamos que el archivo principal ecmp.p4 está correctamente configurado, ya que todos los métodos explicados funcionan correctamente: la máquina de estados, las acciones a realizar del pipeline, el llenado de las tablas, etc.

También comprobamos que una conexión la realiza por el puerto 3 y otra por el 4. Esto nos da una buena indicación de que ambos caminos son elegidos aleatoriamente debido a que ninguno tiene preferencia a otro, lo que nos lleva a pensar que funciona correctamente el balanceo ECMP.

Para comprobarlo correctamente, realizaremos una prueba más exhaustiva en la que emplearemos el archivo send.py que se encargará de enviar tantos peticiones entre *hosts* como queramos (en este caso 1000) para hacer una prueba global de la red, ver Figura 30. Debemos tener en cuenta que para a petición le corresponde una respuesta, por lo que serán 2000 paquetes en total (1000 de ida y 1000 de vuelta).

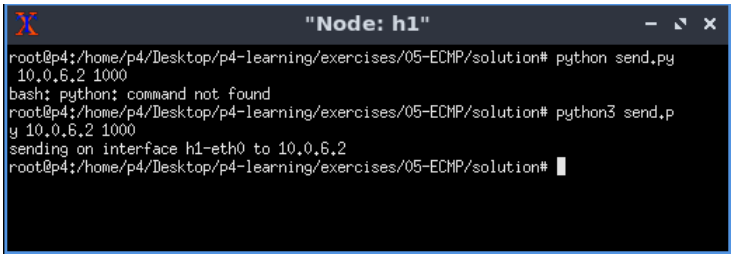


Figura 30. Comando para iniciar una conexión de 1000 peticiones desde H1 hacia H1

Podemos comprobar cada interfaz por separado del primer *switch* para observar que porcentaje de los paquetes circulan por cada uno de los caminos o *switches* intermedios, esperando que se cumpla que aproximadamente, pasen el mismo número de paquetes por cada ruta, cumpliendo así el ECMP. Comprobamos en las Figuras 31, 32, 33 y 34 el comando utilizado y el resultado de los paquetes capturados en una prueba de empleo de send.py.

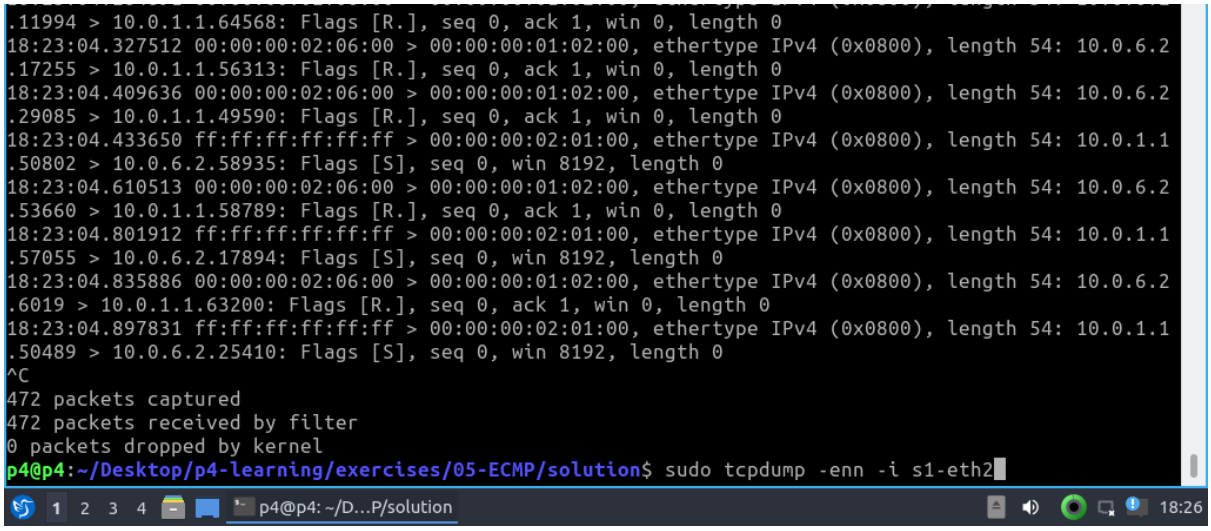


Figura 31. Resultado de observar el enlace s1-s2

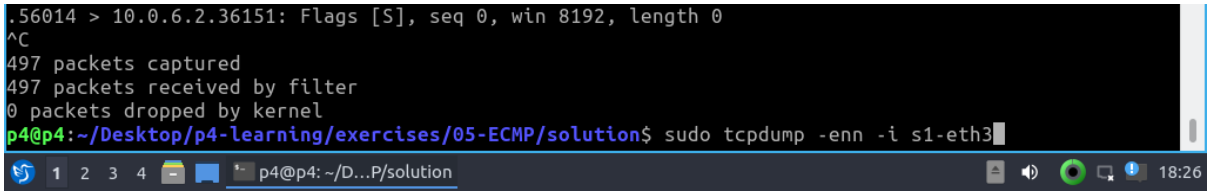


Figura 32. Resultado de observar el enlace s1-s3

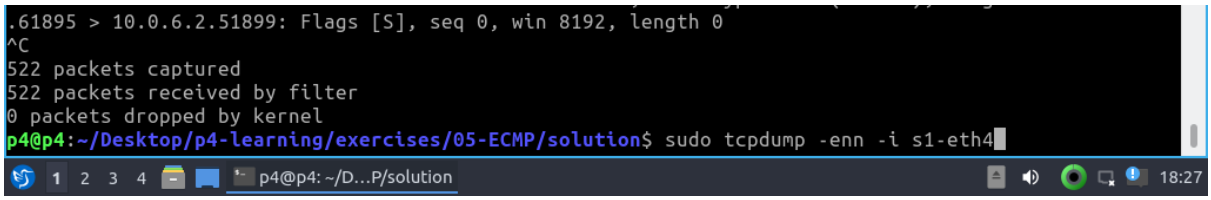


Figura 33. Resultado de observar el enlace s1-s4

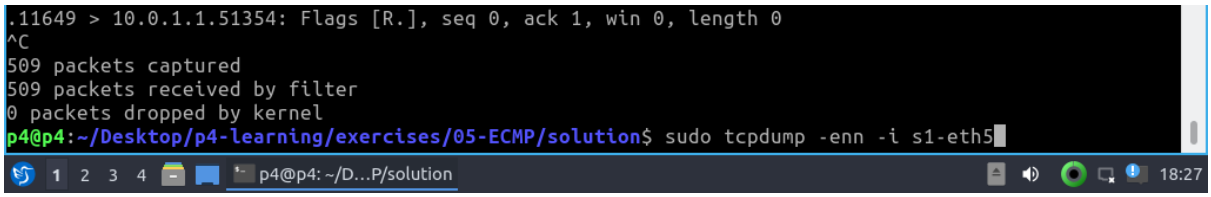


Figura 34. Resultado de observar el enlace s1-s5

En la Figura 35 representamos los resultados del envío hacia H2, de 1000 paquetes SYN (iniciadores de conexión TCP) y 1000 respuestas ACK denegándolas, es decir, 2000 paquetes en total y evaluar cada enlace por separado durante el tiempo de envío de paquetes. Comprobamos que cumple el ECMP esperado:

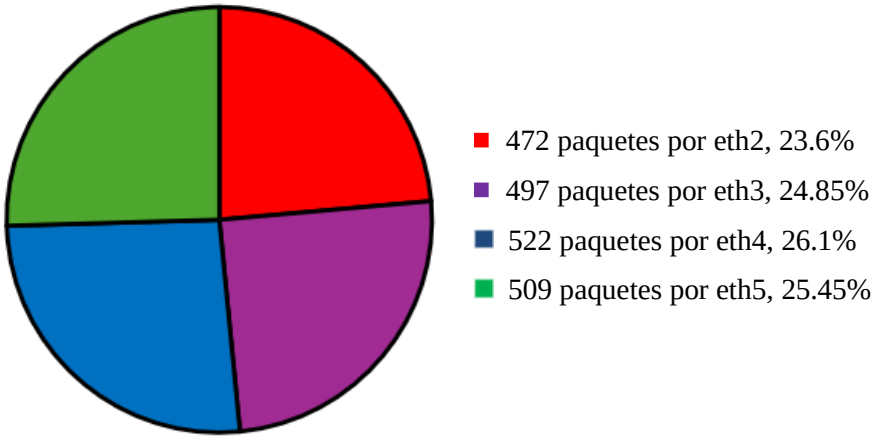


Figura 35. Repartición del tráfico por cada ruta del escenario:

3.3. SEGUNDO ESCENARIO, PRUEBA QUIC

3.3.1. EXPLICACIÓN DE LAS PRUEBAS A REALIZAR

El escenario elegido para el estudio de QUIC se basa en el balanceador de carga más completo encontrado, siendo este: QUIC-LB, propuesto por “erfanw” en un repositorio de GitHub [36].

Las principales inconveniencias de este trabajo son el desfase de los archivos con respecto a las versiones de sus dependencias y la falta de explicaciones en la implementación. Será nuestra tarea realizar todos los cambios pertinentes en el código para su correcta implementación y el entendimiento del código una vez sea completado

Como hemos comentado anteriormente, implementaremos un escenario que genera una topología basada en el trabajo QUIC_LB y representada en la Figura 36, donde visualizamos los elementos de red que lo forman:

- Tres *hosts*: un cliente, el cual mandará peticiones de envío de cierto archivo a dos servidores diferentes.
- Una VIP: una dirección IP virtual asignada a muchas aplicaciones, a la que estará conectado el cliente.
- Un *switch* que funciona como balanceador de carga Round Robin por pesos, que configuraremos con 6 tokens o fichas que distribuiremos entre ambos servidores según la cantidad de tráfico que queramos que genere cada uno de los dos.

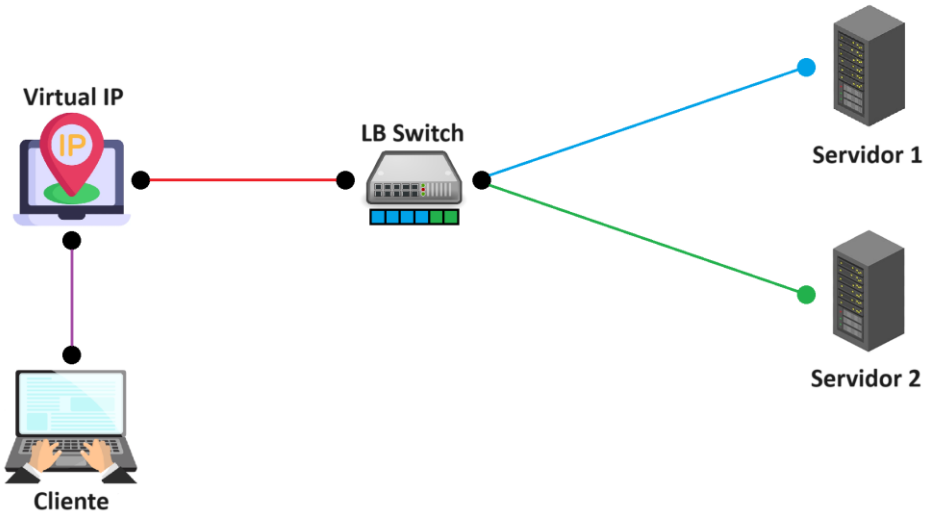


Figura 36. Topología del escenario 2.

En el contexto de este escenario P4, QUIC se emplea con la función de identificar y manejar paquetes que utilizan este protocolo específico dentro de la capa de transporte. El diseño del escenario está particularizado para el empleo de paquetes que usen UDP/QUIC, debido a que no está programado el procesado de los paquetes de otros protocolos, es decir, que no cumplan con el requisito del parseador. QUIC se utiliza para probar la flexibilidad, seguridad y velocidad de la red, que son los principales atributos que podemos asignarle a este protocolo. Esto se logra a través de la implementación de acciones específicas y tablas de búsqueda que gestionan la asignación de servidores y el cálculo del checksum, asegurando así un manejo eficiente y seguro de los paquetes QUIC en el entorno de la red programable

- Procesamiento y Enrutamiento de Paquetes QUIC: El código se encarga de procesar paquetes identificados que empleen el protocolo QUIC sobre UDP en la red. A través del *parser* y las tablas de búsqueda definidas, se extraen y manejan los campos esenciales de los headers, como hdr.udpQuic.srcPort, hdr.udpQuic.hdr_type, y hdr.quicLong.cookie. Estos campos son cruciales para la determinación del servidor destino y la toma de decisiones de encaminamiento eficiente.
- Tablas y Acciones Específicas: Las tablas get_server_from_bucket, get_server_from_id_long_header, y get_server_from_id_short_header son empleadas para asignar los servidores basados en la información extraída de los headers QUIC y otros headers relevantes como IPv4. Estas tablas actúan en conjunto con las acciones definidas, como fwd_to_server, para dirigir los paquetes hacia el servidor adecuado según el contenido de los headers QUIC.
- Seguridad y Eficiencia: QUIC ofrece características avanzadas de seguridad y eficiencia. Utiliza mecanismos integrados de cifrado y autenticación desde el inicio de la conexión, proporcionando una capa adicional de protección y privacidad para las comunicaciones. Esto se logra a través del manejo seguro de los campos hdr.quicLong.cookie y hdr.quicShort.cookie, que son fundamentales para la identificación segura de los servidores y la gestión de sesiones.

La carpeta de archivos que usaremos para la implementación de este escenario tendrá la forma que representa la Figura 37. Al igual que pasaba con el primer escenario, cuenta con un archivo P4 principal, almacenado en p4src, en este caso llamado l2_basic_forwarding.p4, con sus propios includes. Además cuenta con un fichero de configuración CLI para el único *switch* del escenario. La topología del

escenario la proporciona el fichero p4app.json. La estructura y los parámetros empleados en este fichero son muy parecidos a los empleados en el fichero empleado en el escenario TCP, lo cual nos ayudará a realizar la comparación de los escenarios. Como particularidad que posee es el fichero Makefile, que lo único que nos facilitará la adaptación hacia un cliente o un servidor para cada *host*, debido a que en el escenario anterior no era necesario esta distinción. Por último, el fichero crc16.c cuya función es generar un CRC (*Cyclic Redundancy Check*) para detectar errores en datos transmitidos o almacenados

Además de esta carpeta, será necesario usar la carpeta PICOQUIC, representada en la Figura 38 con sus carpetas y ficheros principales, bajo el mismo el directorio que la carpeta de nuestro escenario para que funcione correctamente. Esta carpeta instalada por las dependencias, aparte de las carpetas con los archivos de configuración importantes (picoquic_sample), posee las carpetas client_files y server_files, con el archivo index.txt, que usaremos para comprobar el funcionamiento de la implementación. Al igual que en el escenario anterior, el contenido de este archivo no nos importa demasiado, ya que la comprobación del escenario la basaremos en las peticiones que se realizan para obtener este archivo, no en el propio archivo en sí.

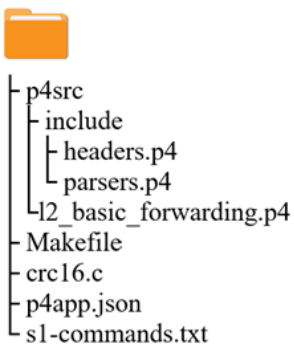


Figura 37. Representación de la carpeta principal para el escenario 2

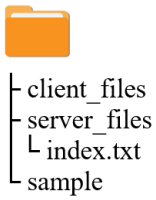


Figura 38. Representación de la carpeta picoquic para el escenario 2

Los códigos de los códigos de la carpeta principal están especificados en el Anexo A.

La topología de este escenario es definida en el archivo p4app.json, en el que además de ella, se definen los módulos Mininet y de topología, donde se emplearán ficheros de la carpeta de utilidades p4-utils, el programa principal empleado l2_basic_forwarding.p4 y demás configuraciones secundarias necesarias para que funcione correctamente la red.

Al igual que en el escenario anterior, se posee un archivo .txt para la configuración de los *switches* CLI. En este caso solo contamos con un *switch*, por lo que tendremos un único fichero pero de mayor complejidad debido a la implementación de los tokens para los servidores para el balanceo de carga. La configuración manual de este fichero será la responsable de balancear la carga del tráfico ya que podremos modificar los pesos para cada servidor. También relacionará la dirección de cada servidor y su id con el puerto de salida correspondiente

Para la configuración P4 de la gestión de los paquetes en la red, contamos con el fichero con la carpeta p4src y el archivo principal l2_basic_forwarfing.p4 con sus respectivos #includes. Este código consta de un parseador, un deparseador (definidos en un fichero parsers.p4, para una mayor limpieza en el código) y de unas instrucciones en el *pipeline* de los datos entre las que constituyen principalmente las entradas *Ingress* y *Egress*, para construir virtualmente los *switches*. Además, al ejecutar la creación de la red se generarán dos archivos más con el mismo nombre en formato l2_basic_forwarfing.json y l2_basic_forwarfing.p4i.

PARSER (Código anexado, parser.p4). Se define una máquina de estados finita representada en la Figura 39, en la que cada estado extrae su propia cabecera de los paquetes, las guarda en variables definidas (las cabeceras necesarias también son definidas en otro fichero headers.p4) y cambia al siguiente dependiendo del valor del encabezado. Los primeros estados son iguales al parseador de TCP, sin embargo, el cambio se produce en los estados específicos de QUIC.

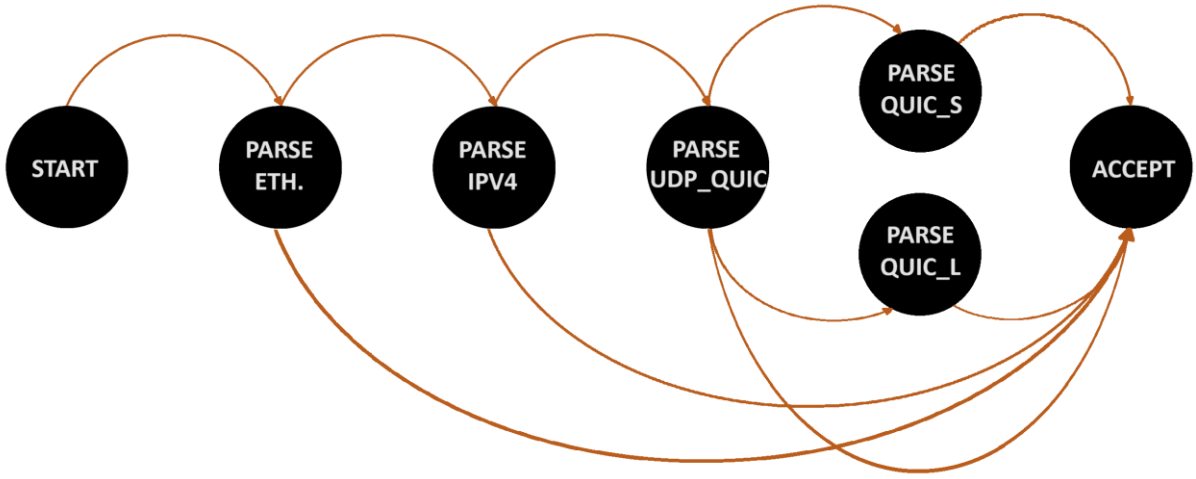


Figura 39. Representación de la máquina de estados del *Parser* QUIC

Start. Estado inicial, cambia directemente hacia el estado de Ethernet.

Parse Ethernet. Extrae y almacena la cabecera Ethernet (hdr.ethernet) del paquete que contiene campos como srcAddr, destAddr y etherType. Si el campo etherType (hdr.ethernet.etherType) es de tipo IPv4, cambia al estado de IPv4, si contiene cualquier otro valor, completa el procesamiento del paquete cambiando hacia el estado final.

Parse IPv4. Extrae y almacena la cabecera IPv4 del paquete en la estructura hdr.ipv4. Si el campo protocol (hdr.ipv4.protocol) de esa cabecera es “17”, es decir que use el protocolo UDP, cambia al estado de UDP_QUIC, si contiene cualquier otro valor, completa el procesamiento del paquete cambiando hacia el estado final.

Parse UDP_QUIC. Extrae la cabecera UDP_QUIC y comprueba su tipo. Si su tipo es “0” cambia al estado quicShort, si por el contrario es “1” cambia al estado quicLong. Se configura también que si no cumple ninguno de los dos casos, por defecto cambie al estado final.

Parse QUIC_SHORT. Extrae la cabecera correspondiente y la almacena en la estructura hdr.quicShort. Tras esto cambia al estado final.

Parse QUIC_LONG. Extrae la cabecera correspondiente y la almacena en la estructura hdr.quicLong. Tras esto cambia al estado final.

Accept. Estado final, finaliza la función del *Parser* concluyendo el procesamiento del paquete.

DEPARSER (Código anexado, parser.p4). Se limita a agregar las cabeceras extraídas a los paquetes procesados para su salida del *switch*. Si el paquete ha completado todos los estados de la máquina, serán : hdr.ethernet, hdr.ipv4, hdr.udpQuic, hdr.quicShort y hdr.quicLong. Como hemos comentado previamente, la definición de la arquitectura de las cabeceras y las estructuras están definidas en el fichero headres.p4, en la carpeta /include.

PIPELINE (Código anexado, ecmp.p4). Consta de todos los demás módulos que constituyen el *switch*.

- **VerifyCheckSum.** En el caso de este escenario, al igual que el anterior, no es necesario realizar ninguna verificación con las cabeceras ni con los metadatos para la integridad de los datos.
- **Ingress.** Esta sección es la más importante del programa, siendo así también la más extensa. Instruye las acciones a realizar por el programa:
    - crc_hash. Almacena en un metadato llamado crc16 el resultado de un cálculo *hash* empleando el algoritmo crc16.c con unos datos introducidos que especifican el puerto y la dirección de un paquete. Este código de cálculo CRC16 es un implementación útil y bastante usada para la detección de errores en transmisiones de datos.
    - fwd_to_server. Modifica ciertos campos en el encabezado del paquete, (la dirección IP, DIP y MAC, DMAC de destino) y define el puerto de salida (*egress port*) para dirigir el paquete hacia un servidor específico. Todas estas modificaciones se realizan actualizando los campos a los valores de las cabeceras parseadas previamente.

  Define las tablas que serán necesarias para el encaminamiento de los paquetes en los *routers*:

    - get_server_from_bucket: Determina a qué servidor debe enviarse un paquete basándose en un identificador de "*bucket*" (cubo) observando el metadato bucket_id.
    - get_server_from_id_long_header: Se encarga de redirigir paquetes a servidores específicos basándose en un identificador de servidor (server_id) que se encuentra en la metadata del paquete. Este server_id tendrá la forma de id largo, por lo que será empleado en el estado QUIC_Long del *parser*.
    - get_server_from_id_short_header: Tiene la misma función que la tabla anterior pero está diseñado para emplear el id corto de servidor (server_id) en el estado QUIC_Short del *parser*.

  Además especifica ciertas acciones aritméticas secundarias empleadas para el cálculo del checksum y validación de los paquetes.

    - ones_complement_sum: Realiza la suma de dos números de 16 bits usando el complemento a uno.
    - subtract: Realiza la resta de un número de 16 bits (d) de sum usando el complemento a uno.
    - subtract32: Realiza la resta de un número de 32 bits (d) de sum usando el complemento a uno.
    - add: Realiza la suma de un número de 16 bits (d) a sum usando el complemento a uno.
    - add32: Realiza la suma de un número de 32 bits (d) a sum usando el complemento a uno.

  En el bloque apply, el código tiene varias funciones:

  Primero realiza *checksum* de manera manual, inicializa sum a 0 y, empleando las funciones aritméticas subtract y subtract32, definidas previamente, resta ciertos valores: el checksum UDP QUIC actual (hdr.udpQuic.checksum), la dirección IP de destino (hdr.ipv4.dstAddr), la dirección IP de origen (hdr.ipv4.srcAddr) y la longitud del paquete UDP QUIC (hdr.udpQuic.length).

Seguido a eso, realiza el procesamiento de los paquetes QUIC (solo procesa los paquetes que tienen un encabezado UDP QUIC válido), verifica si el paquete es de un cliente (“ipv4.dstAddr” dirigido a una IP virtual) y maneja las conexiones iniciales dependiendo si el paquete consta de un encabezado largo o corto. Si el paquete procesado consta de encabezado largo (“udpQuic.hdr_type” = 1), Lee el contador de "*bucket*" y lo almacena en el metadato “bucket_id”; busca y redirige el paquete hacia un servidor basado en el bucket_id (get_server_from_bucket.apply()) y por último actualiza el contador de "*bucket*”. Tras esto, calcula un *hash* CRC con el puerto de origen y la dirección IP de origen, Combina el *hash* con una *cookie* específica del encabezado (corto o largo) para obtener “server_id” y usa ese metadato obtenido para redirigir el paquete al servidor adecuado.

Aparte a eso, en el caso de que se trate de un paquete enviado por alguno de los servidores, se redirige al cliente de forma básica: Se ajusta la dirección IP de origen al “virtual_ip”, se ajusta la dirección MAC de destino al “client_mac” y se especifica el puerto de salida “egress_spec”

Por último, ajusta y recalcula el checksum UDP QUIC al final del procesamiento del paquete. Emplea las funciones de add y add32 para sumar las nuevas cabeceras udpQuic.length, ipv4.srcAddr y ipv4.dstAddr al valor sum inicial. Tras eso, asigna el complemento a uno del `sum` como el nuevo checksum UDP QUIC: hdr.udpQuic.checksum = ~sum.

- **Egress.** El procesamiento de *egress* se realiza justo antes de que un paquete salga del *switch* y permite realizar cualquier modificación necesaria en los encabezados de los paquetes. En este caso, *egress* está vacío ya que no es necesario ningún cambio en las cabeceras de los paquetes procesados.

- **ComputeCheckSum.** Si el encabezado IPV4 es válido, actualiza el checksum de este encabezado (hdr.ipv4.hdrChecksum) empleando el algoritmo csum16. Esto se realiza para asegurar que los encabezados de los paquetes sean correctos antes de su salida del Pipeline y su envío posterior.

CONSTRUCCION DEL *SWITCH*. Al igual que en escenario anterior, se genera la arquitectura requerida para el *switch* programable (arquitectura PISA) V1Switch con los módulos descritos previamente: MyParser(), MyVerifyChecksum(), MyIngress(), MyEgress(), MyComputeChecksum(), MyDeparser().

3.3.2. RESULTADOS DE RENDIMIENTO

Al tener los *logs* de la red activados, podemos comprobar el rellenado de las tablas del *switch* en la Figura 41. Esto nos dará la idea de que las redes estarán bien configuradas en el reenvío de los paquetes y que las tablas programadas en el fichero P4 cumplen su labor correctamente.

Tras comprobar el correcto funcionamiento de la red, creación, conexión, balanceo de tráfico, etc. y con ello, entender el papel que ha desempeñado QUIC en la implementación de P4, los resultados que pretendemos conseguir con la implementación de este escenario basado en QUIC son:

Comprobar la velocidad de la red, comprobando que para cada petición no se requiere más de un mensaje, observando que el protocolo cumple el handshake 0-RTT.

Comprobar que las comunicaciones solo permiten paquetes UDP/QUIC, comprobando la exclusividad del escenario, y que estas se dan de manera segura, para esto podemos comprobar los *logs*

que dejan los paquetes encriptados para las conexiones QUIC y el error de transmisión para las conexiones distintas a UDP/QUIC (ICMP en este caso, realizadas con un ping).

Comenzaremos corroborando que se genera la Mininet de la red correctamente utilizando los mensajes de topologías y creaciones de elementos como comprobante, analizando que son correctos y coherentes al escenario empleado. Esto podemos comprobarlo en la Figura 40.

Seguidamente comprobaremos el correcto llenado de las tablas y por ende, las funcionalidades de las acciones del código P4.

Posteriormente, evaluaremos la conexión dependiendo de los tipos de paquetes entre el cliente y los servidores en las figuras siguientes.

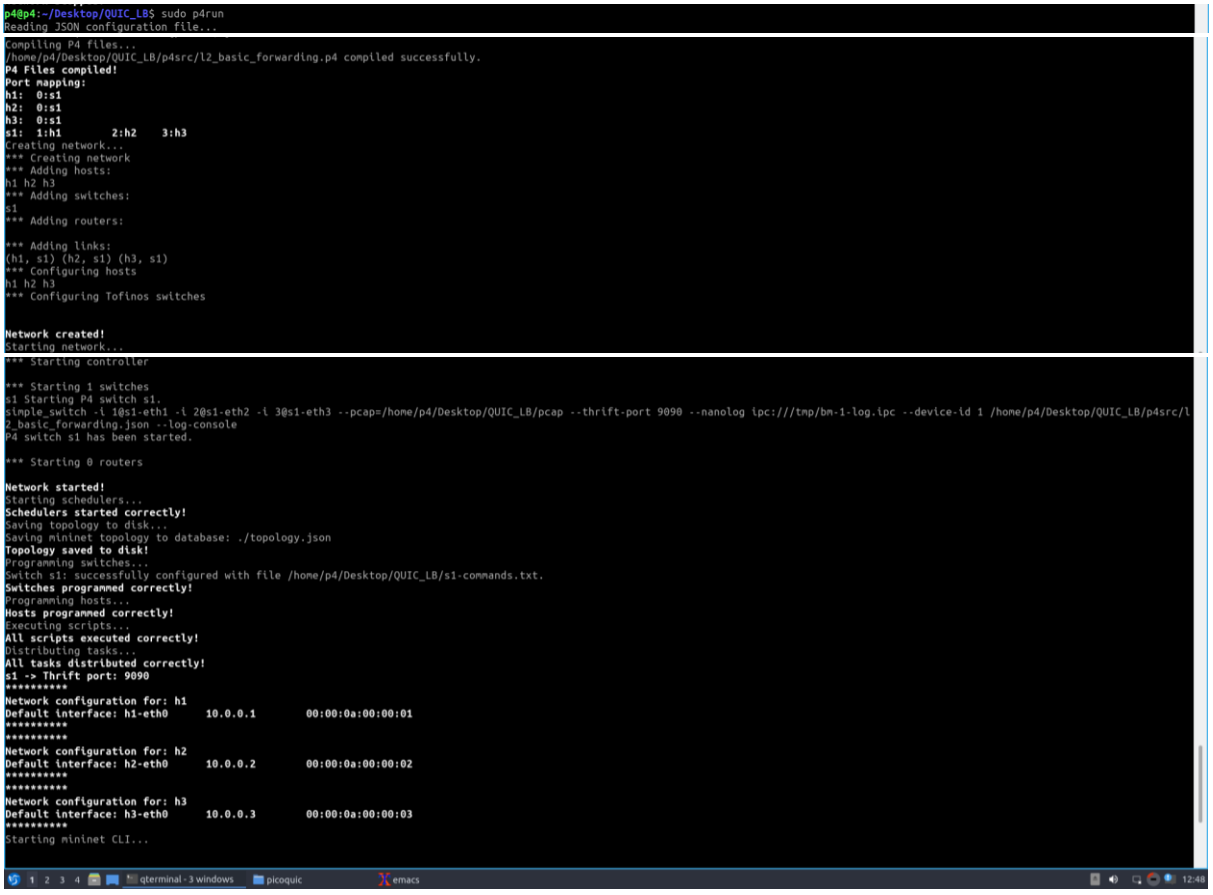


Figura 40. Creación correcta de la Mininet del escenario 2

Evaluaremos la transmisión de las peticiones del cliente (10.0.0.1) hacia los servidores (10.0.0.2 y 10.0.0.3). En la siguiente Figura 41 podemos comprobar que se envían primero cuatro peticiones hacia Server2, seguidas de dos hacia Server3, confirmando correctamente la distribución de pesos y el correcto orden de reparto de los *tokens* en el "cubo" de servidores como hemos configurado en el archivo s1-commands.txt.

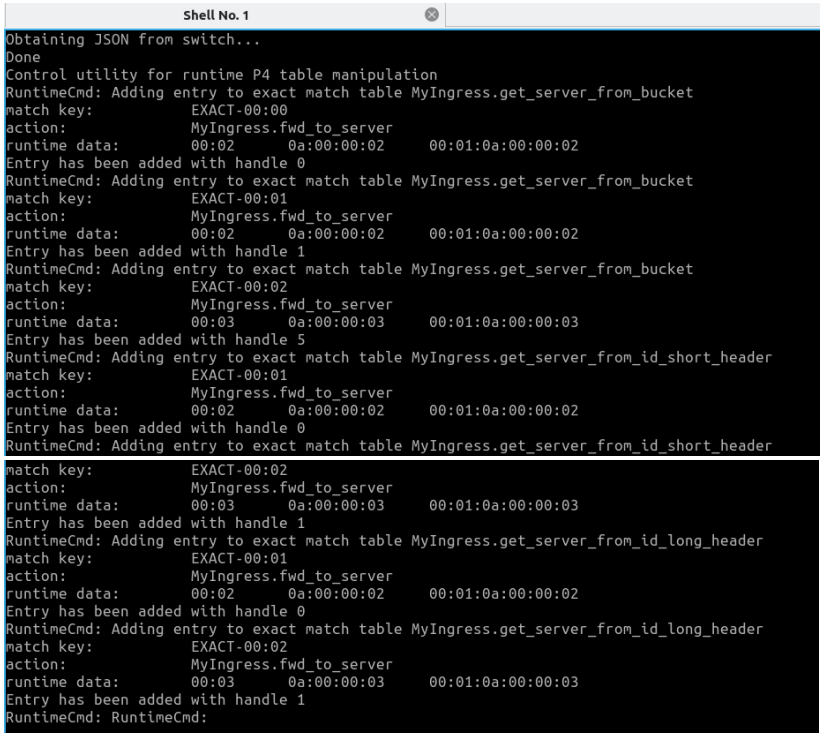


Figura 41. Configuración de las tablas del *switch* del escenario 2

Estudiando la captura de paquetes generada por Wireshark, visible en la Figura 42, podemos comprobar mucha información útil del paquete reenviado. Entre ella, algunos campos que nos indican que se cumplen los requisitos del parseador para que el procesado del paquete pase por todos los estados hasta finalizar: Ethernet Type = IPv4 y IPv4 *protocol* = 17 (UDP). Sin embargo, estas capturas de Wireshark no nos aportan la información necesaria para comprobar si el paquete emplea una cabecera larga o corta, por lo que será comprobado mediante la revisión de los *logs* que genera el *switch* cuando el paquete atraviese su parseador, reflejados en la Figura 43.

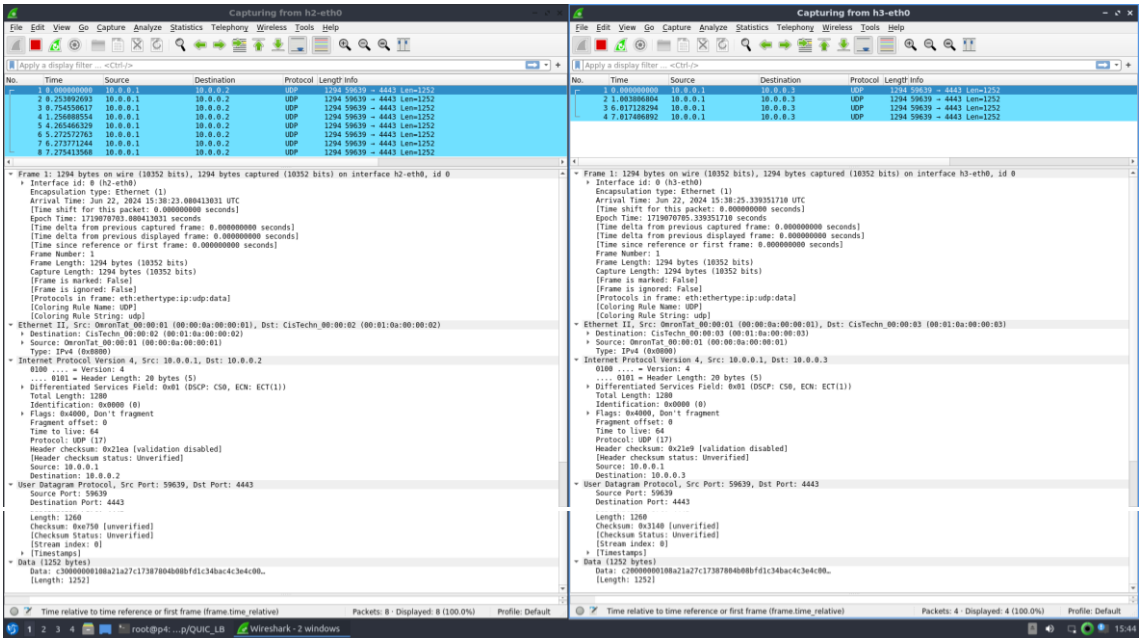


Figura 42. Captura de las solicitudes a los servidores.

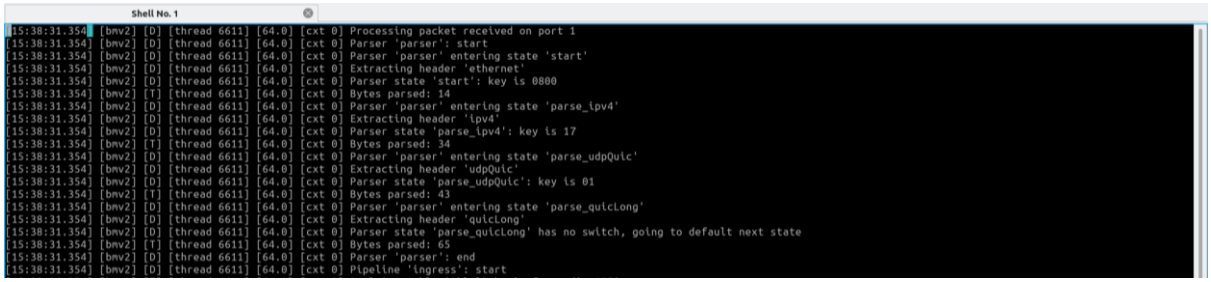


Figura 43. Trazas del *parser* del *switch*

En la Figura 44 siguiente comprobamos que los *hosts* están conectados con el *switch* (hx ping s1 es fructífero), es decir, los enlaces funcionan correctamente, pero el *switch* está configurado para no procesar archivos que no sigan el protocolo QUIC, por lo que la herramienta ping entre *hosts* (hx ping hx pierde el paquete enviado), que emplea un protocolo ICMP, no genera conexión entre los *hosts* debido a que no son reenviados por el *switch* intermedio.

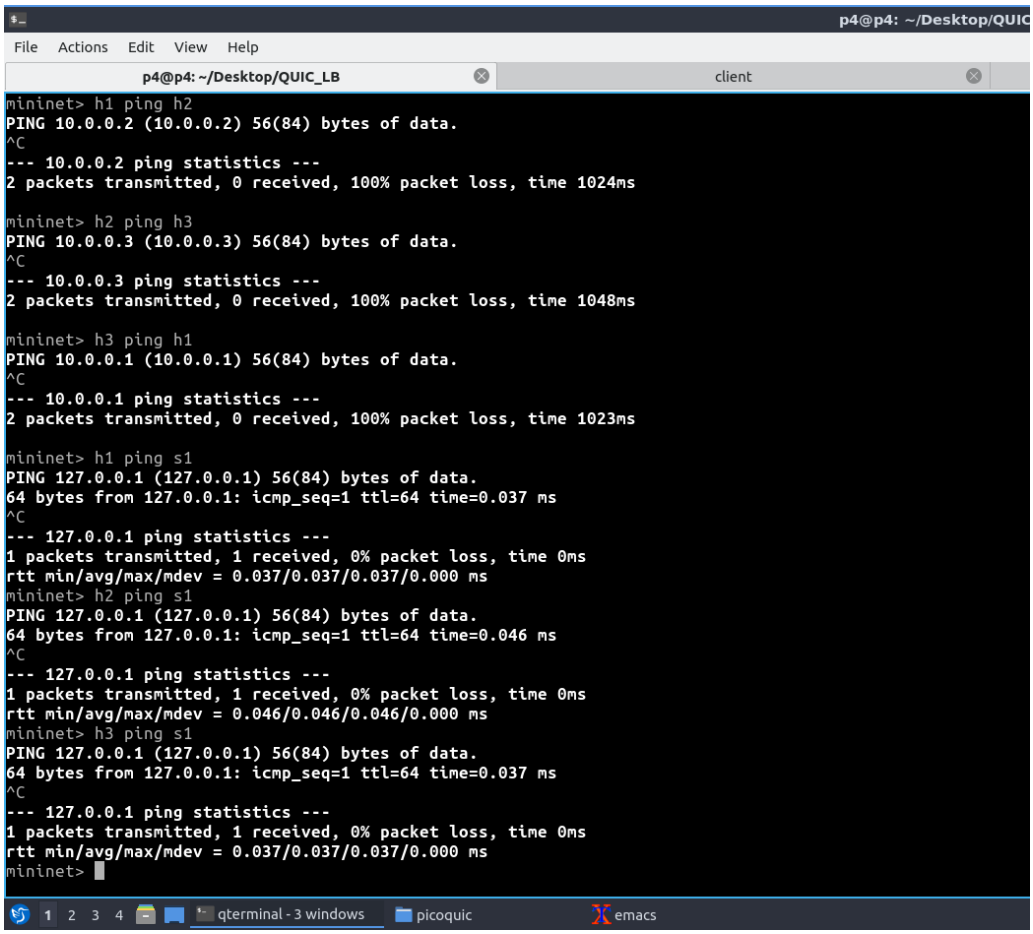


Figura 44. Pruebas de *ping* entre los elementos de la red

Comprobamos que el paquete se distingue por el ConnectionID inicial que se genera automáticamente al iniciar la conexión del cliente hacia la VIP, addr_to: 10.0.0.254 por el puerto: 4443, visible en la Figura 45. Como el escenario está configurado para analizar las peticiones, no llega a recibir los archivos desde los servidores que están pensados como elementos de almacenamiento. Para el transcurso de la conexión, analizando las trazas que dejan las solicitudes del cliente representadas en la Figura 46, vemos como se genera un ConnectionID de la conexión diferente al original, en este caso: hadshake_connection_id: "a7dfa94bd02f46a3", que se comparte con el servidor, y varios parámetros más de la configuración de conexión.

Comprobamos que hay parte del fichero transmitido que se encuentra encriptado, frame_type: crypto.

La longitud del paquete concuerda con la captura de Wireshark, Len: 1252 de los datos, con el apartado byte_length : 1252 de los *logs*.

```
p4@p4:~/Desktop/QUIC_LB$ mx h1
root@p4:/home/p4/Desktop/QUIC_LB# make client
arp -s 10.0.0.254 a1:23:e3:8d:4b:01 && cd ../picoquic && ./picoquic_sample client 10.0.0.254 4443 ./client_files index.html
Starting connection to 10.0.0.254, port 4443
Initial connection ID: 1ce05038f71747fd
Opened stream 0 for file index.html
Waiting for packets.
Connection closed.
index.html: unknown status, received 0 bytes
root@p4:/home/p4/Desktop/QUIC_LB# ^C
root@p4:/home/p4/Desktop/QUIC_LB#
```

Figura 45. Configuración de cliente solicitante en el *host*

```
Shell No. 1
{ "qlog_version": "draft-00", "title": "picoquic", "traces": [
{ "vantage_point": { "name": "backend-67", "type": "client" },
"title": "picoquic", "description": "1ce05038f71747fd","event_fields": ["relative_time", "category", "event", "data"],
"configuration": {"time_units": "us"},
"common_fields": { "protocol_type": "QUIC_HTTP3", "reference_time": "70691009729"},
"events": [
[2198, "transport", "parameters_set", {
    "owner": "local",
    "sni": "test.example.com",
    "proposed_alpn": ["picoquic_sample"]}],
[2201, "transport", "parameters_set", {
    "owner": "local",
    "initial_max_stream_data_bidi_local" : 2097152,
    "initial_max_data" : 1048576,
    "initial_max_streams_bidi" : 512,
    "idle_timeout" : 30000,
    "max_packet_size" : 1440,
    "initial_max_streams_uni" : 512,
    "handshake_connection_id"bidi_remote" : 65635,
    "initial_max_stream_data_uni" : 65535,
    "active_connection_id_limit" : 8,
[2533max_ack_delay" : 10,
f    handshake_connection_id": "a7dfa94bd02f46a3",
     enable_loss_bit" : 1,
    "min_ack_delay" : 1000}],
[
46a3", "dcid": "1ce05038f71747fd" }, "frames": [{
    "frame_type": "ping"}, {
    "frame_type": "crypto", "offset": 0, "length": 287}, {
    "frame_type": "padding"}]}],
[2533, "transport", "datagram_sent", { "byte_length": 1252, "addr_to" : {"ip_v4": "10.0.0.254", "port_v4":4443}, "addr_
[305680, "recovery", "packet_lost", {
    "packet_type" : "initial",
    "packet_number" : 113113,
    "trigger": "timer",
    "header": {
        "packet_type" : "initial",
        "packet_number" : 113113,
        "packet_size" : 1236}}],
1 2 3 4 qterminal - 3 windows picoquic XTerm - 3 windows
```

Figura 46. Trazas de las solicitudes UDP/QUIC del cliente

3.4. COMPARACIÓN DE LOS ESCENARIOS ESTUDIADOS

Ambos escenarios implementan correctamente el balanceo de cargas. Podemos afirmar que TCP es un protocolo más adecuado para aplicaciones que requieren fiabilidad y entrega ordenada de datos, como transferencia de archivos y aplicaciones empresariales, pero genera una gestión más compleja de conexiones y control de flujo en comparación con UDP/QUIC. Este, en cambio, puede ser más adecuado para aplicaciones que requieren baja latencia y conexiones seguras, como aplicaciones web y servicios de transmisión.

Las ventajas que puede aportar QUIC es el manejo de Checksums personalizados, la posibilidad de diferenciar los paquetes iniciales de los no iniciales, con el fin de tratarlos de maneras diferentes (los iniciales se redirigen utilizando un sistema de contadores de "*bucket*" para equilibrar la carga entre servidores y los no principales se utilizan identificadores de servidor (server_id) generados mediante un *hash* CRC y *cookies* específicas del encabezado QUIC para dirigir los paquetes al servidor correcto). Esto hace que no todos los paquetes tengan la misma complejidad de balanceo, cargando de menor tráfico el procesado de los paquetes.

Vemos en el código como QUIC funciona sobre UDP y proporciona características avanzadas como el establecimiento de conexiones más rápido, multiplexación de múltiples flujos en una única conexión, y manejo eficiente de pérdidas de paquetes. TCP, en cambio, es un protocolo basado en conexión que garantiza la entrega ordenada pero más lenta de paquetes.

Los encabezados propios de TCP y QUIC son diferentes y tienen campos específicos como hdr_type y pkt_type, así como *cookies* de encabezado largo y corto.

QUIC no requiere el establecimiento de conexión en tres pasos (*three-way handshake*) como TCP, lo cual simplifica y acelera el proceso de inicio de conexiones. Este código maneja la asignación de servidores y la redirección de una manera optimizada para QUIC. Podemos comprobar en las Figuras 27 y 42 la comparativa del establecimiento de conexión para ambos protocolos:

Petición de 1 paquete en TCP:

1. SYN: Paquete de inicio de la conexión TCP (paquete de sincronización).
2. SYN-ACK: Paquete de respuesta al SYN, indicando aceptación de la conexión.
3. ACK: Paquete de confirmación de recibo.
4. FIN-ACK: Paquete de finalización de la conexión
5. ACK: Paquete de confirmación de la finalización de la conexión.

Petición de 1 paquete en QUIC

1. QUIC: Paquete de petición del archivo

QUIC realiza la conexión con el ConnectionID del paquete, siendo este visible, por lo que para conexiones “relevantes” se encripta el contenido de los *logs* producidos por el paquete empleando OpenSSL para denegar la capacidad de compartición de contenidos. TCP al contrario, como necesita esa doble verificación, no necesita encriptar los archivos porque se prevé una conexión más privada empleando la técnica de confirmación de mensajes en la comunicación, ACK.

3.5. POSIBLES APLICACIONES EN 5G

El escenario estudiado basado en QUIC brinda otras facultades a implementar en las redes 5G. Una implementación interesante puede ser en la telemetría en banda (INT). Esta aplicación es inducida debido a la capacidad de QUIC para ofrecer conexiones seguras y rápidas (ideal para aplicaciones que requieren baja latencia y alto rendimiento, como en este caso, sensibles a la latencia como INT) y a su capacidad de medir en tiempo real parámetros de rendimiento como el RTT, la latencia y la pérdida de paquetes.

QUIC integra cifrado de extremo a extremo y autenticación, proporcionando una capa de seguridad adicional, cumpliendo el requisito de la seguridad de conexión. Además, optimiza el establecimiento de conexiones y la transferencia de datos, reduciendo la latencia y mejorando el rendimiento general. Integrando este protocolo con QUIC, se podrían insertar metadatos en los paquetes, definidos en los códigos P4, para medir parámetros de rendimiento sin afectar significativamente el flujo de datos y sin desproteger las mediciones, ya que se transportarían cifradas y autenticadas.

Otra posible aplicación para el protocolo QUIC es en términos de ciberseguridad, ya que hemos comprobado que uno de los ámbitos en los que más destaca QUIC es en la seguridad de su transmisión ya que los paquetes normales o estándar se encuentran autenticados y encriptados (excluyendo la cabecera pública) de manera automática y por defecto, lo que deniega la capacidad de inyección o de manipulación de datos por obra de terceros.

Además de esto, hemos comprobado como la velocidad de conexión entre los extremos de la conexión es mayor empleando este protocolo, lo cual sería necesario para soportar las capacidades requeridas por la red 5G. Esto hace que la capa de seguridad en el transporte de los paquetes generada no reduzca la latencia de la conexión cumplimentando los objetivos de latencias que tiene 5G.

4. CONCLUSIONES Y LÍNEAS FUTURAS

4.1. CONCLUSIONES

El objetivo principal de este estudio trata de implementar, evaluar y comparar el protocolo TCP y QUIC en el contexto de arquitecturas 5G, utilizando el plano de datos programable y el lenguaje de programación P4 como tecnologías habilitantes. A continuación, se detalla cómo se han cumplido cada uno de los subobjetivos planteados:

En el Capítulo 2 se ha realizado un estudio exhaustivo de las tecnologías SDN, el plano de datos programable, la arquitectura PISA, el lenguaje P4, así como los casos de uso para 5G. Además, se exploraron los fundamentos teóricos de QUIC y TCP, proporcionando una base sólida para la comparación y evaluación posterior.

En el Capítulo 3 del trabajo se han llevado a estudios detallados para identificar los escenarios y arquitecturas óptimas para la implementación de balanceadores de carga utilizando tanto TCP como QUIC. Esto incluyó consideraciones sobre la eficiencia, la escalabilidad y la flexibilidad de cada protocolo en entornos SDN aplicados a redes 5G.

Posteriormente, se muestra cómo han sido desplegadas arquitecturas SDN para la implementación práctica de balanceadores de carga utilizando TCP y QUIC. Este paso permitió evaluar el desempeño real de cada protocolo en condiciones controladas y simular diferentes cargas de trabajo y condiciones de red.

En el Capítulo 4 se han evaluado de manera exhaustiva los escenarios implementados, centrándolo en la comparación de TCP y QUIC en términos de rendimiento, seguridad y adaptabilidad a condiciones de red variables. Las pruebas demostraron que QUIC supera a TCP en eficiencia gracias a su capacidad de multiplexación, establecimiento rápido de conexiones y adaptabilidad a pérdidas de paquetes y cambios de red, mientras que TCP tiene la ventaja de ser un protocolo más fiable, estable y utilizado en un mayor porcentaje de las implementaciones actuales.

En resolutiva, tras recoger los resultados más relevantes de los distintos escenarios, comprobamos que ambos protocolos poseen cualidades positivas y frente al otro y viceversa. Mientras que por un lado, QUIC ofrece numerosas ventajas en términos de rendimiento, seguridad y flexibilidad, TCP sigue siendo una opción viable y preferida en muchos entornos donde la estabilidad, la previsibilidad y la compatibilidad son prioritarias. La elección entre TCP y QUIC dependerá de las necesidades específicas de la aplicación, las características de la red y los requisitos de rendimiento y seguridad.

Es visible que una de las grandes virtudes que posee TCP es que en la actualidad está plenamente asentado y muchas funcionalidades de los dispositivos son coherentes con este protocolo y no con otros. Esto nos hace pensar que quizás, para un desarrollo de un escenario que requiera una mayor fiabilidad o seguridad en su desarrollo sea mejor su uso. Por otra parte, también apreciamos como las características que aporta QUIC son una mejora para el transporte de los paquetes de la red, por lo que si pensamos en un escenario menos desarrollado con objetivos de pleno funcionamiento más a futuro, es una opción muy para tener en cuenta, ya que su estudio cada vez se está tomando más importancia y sus bases están cada vez más asentadas.

Estas conclusiones destacan la importancia de adaptar los protocolos de transporte a las demandas cambiantes de las redes modernas, asegurando así un rendimiento óptimo y una experiencia de usuario mejorada en el contexto de las arquitecturas 5G y más allá.

4.2. LÍNEAS FUTURAS

Partiendo del presente trabajo y del software descrito, disponible en GitHub, se proponen las siguientes líneas futuras:

- Modificación del escenario de TCP para la construcción de un balanceo de carga por pesos, una vez los códigos QUIC sean más completos y ofrezcan escenarios más complejos para un estudio más exhaustivo. Esto puede servir para una comparación y conclusión más pulida al tratarse de dos escenarios parecidos
- Concluir el escenario de QUIC para observar la transmisión de paquetes en una conexión en tiempo real más similar a un caso natural de uso, completando el envío de los archivos solicitados, contemplando todos los paquetes en la conexión.

5. BIBLIOGRAFÍA

[1] L. S. Velázquez, «EVALUACIÓN DE LOS DATOS PROGRAMABLES Y SU APLICACIÓN A ENTORNOS 5G,» 2022. [En línea]. Available: https://www.programa5gfranciscoros.etsit.upm.es/?p=1613.

[2] cisco, «Software-Defined Networking,» [En línea]. Available: https://www.cisco.com/c/en/us/solutions/software-defined-networking/overview.html.

[3] ,. M. H. D. M. S. L. V. G. F. Z. R. F. M. M. Frederik Hausera, «A Survey on Data Plane Programming with P4: Fundamentals, Advances, and Applied Research,» 4 Agosto 2021. [En línea]. Available: https://arxiv.org/pdf/2101.10632.

[4] «OpenFlow Switch Specification,» 31 Diciembre 2009. [En línea]. Available: https://opennetworking.org/wp-content/uploads/2013/04/openflow-spec-v1.0.0.pdf.

[5] O. Gerometta, «Planos de operación de un dispositivo de red - Gráfica,» 27 Agosto 2017. [En línea]. Available: https://librosnetworking.blogspot.com/2017/08/planos-de-operacion-de-un-dispositivo.html.

[6] tomy.tim, «Cisco SDN: Control Plane e Data Plane,» 22 Julio 2022. [En línea]. Available: https://community.cisco.com/t5/blogs-routing-y-switching/cisco-sdn-control-plane-e-data-plane/ba-p/4655704#:~:text=Por%20concepto%3A,a%20trav%C3%A9s%20de%20un%20dispositivo.

[7] V. Gurevich, «Programmable Data Plane at Terabit Speeds,» 16 Mayo 2017. [En línea]. Available: https://opennetworking.org/wp-content/uploads/2020/12/p4_d2_2017_programmable_data_plane_at_terabit_speeds.pdf.

[8] P4.org, «P4 Languaje Tutorial,» 22 Septiembre 2022. [En línea]. Available: https://es.slideshare.net/slideshow/p4tutorialpdf/253114324.

[9] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese y D. Walker, «Programming Protocol-Independent,» ACM SIGCOMM, 3 Julio 2014. [En línea]. Available: https://dl.acm.org/doi/pdf/10.1145/2656877.2656890.

[10] A. F. Nate Foster, «P4 Becomes an Independent Project, Hosted by the Linux Foundation,» 4 Enero 2024. [En línea]. Available: https://opennetworking.org/news-and-events/blog/p4-becomes-an-independent-project-hosted-by-the-linux-foundation/.

[11] A. L. A. R. A. W. A. V. C. K. D. Z. F. Y. F. K. . I. S. . J. I. J. B. J. D. . J. R. J. K. P. W. R. T. R. S. R. H. V. V. V. V. y Z. S. , «The QUIC Transport Protocol:,» 25 Agosto 2017. [En línea]. Available: https://dl.acm.org/doi/pdf/10.1145/3098822.3098842.

[12] A. Crespo, «QUIC, el protocolo de Google que promete un Internet más rápido,» 19 Abril 2015. [En línea]. Available: https://www.redeszone.net/noticias/redes/uso-nube-trabajar-estudiar/.

[13] Á. D. Bocos, «COMPORTAMIENTO DEL PROTOCOLO QUIC SOBRE CANALES INALÁMBRICOS,» Septiembre 2019. [En línea]. Available: https://repositorio.unican.es/xmlui/bitstream/handle/10902/16943/419294.pdf?sequence=1&isAllowed=y.

[14] W3Techs, «Usage statistics of HTTP/3 for websites,» 22 Mayo 2024. [En línea]. Available: https://w3techs.com/technologies/details/ce-http3.

[15] W3Tech, «Usage statistics of HTTP/2 for websites,» 22 Mayo 2024. [En línea]. Available: https://w3techs.com/technologies/details/ce-http2.

[16] axarnet, «Qué es el HTTP: Futuro del protocolo【HTTP/3 y QUIC】,» [En línea]. Available: https://axarnet.es/blog/http-futuro-http3-quic. [Último acceso: 25 Junio 2024].

[17] «112 – Una nueva WWW, más rápida,» 28 Septiembre 2019. [En línea]. Available: https://elgatodeturing.com/tag/naukas/.

[18] IONOS, «QUIC: los entresijos del protocolo experimental de Google,» 1 Marzo 2023. [En línea]. Available: https://www.ionos.es/digitalguide/hosting/cuestiones-tecnicas/quic-protocolo-de-transporte-de-internet-basado-en-udp/.

[19] G. Huston, «Comparing TCP and QUIC,» Noviembre 2022. [En línea]. Available: https://www.potaroo.net/ispcol/2022-11/quicvtcp.html.

[20] WIKIPEDIA, «QUIC,» 24 Abril 2024. [En línea]. Available: https://es.wikipedia.org/wiki/QUIC.

[21] J. S. R. Guillermo Goizueta, «La evolución del Data Center en 2017,» 2 Noviembre 2017. [En línea]. Available: https://red.computerworld.es/actualidad/la-evolucion-del-data-center-en-2017.

[22] Topologías de redes, «¿Qué es el balanceo de carga en redes?,» [En línea]. Available: https://topologiasdered.com/dispositivo/balanceo-carga-redes/.

[23] Byte, «Balanceo de carga,» 17 Abril 2019. [En línea]. Available: https://redesteleco.com/balanceo_de_carga/.

[24] 3GPP, «Release 15,» 26 Abril 2019. [En línea]. Available: https://www.3gpp.org/specifications-technologies/releases/release-15.

[25] UIT-R, «Concepción de las IMT – Marco y objetivos generales del futuro desarrollo de las IMT para 2020 y en adelante,» Septiembre 2015. [En línea]. Available: https://www.itu.int/dms_pubrec/itu-r/rec/m/R-REC-M.2083-0-201509-I!!PDF-S.pdf.

[26] 3GPP, «System architecture for the 5G System (5GS) (3GPP TS 23.501 version 17.5.0 Release 17),» Julio 2022. [En línea]. Available: https://www.etsi.org/deliver/etsi_ts/123500_123599/123501/17.05.00_60/ts_123501v170500p.pdf.

[27] G. Mitchell, «Background to NSA,» 29 noviembre 2018. [En línea]. Available: https://www.mpirical.com/blog/background-to-nsa.

[28] H. Remmert, «¿Qué es la arquitectura de red 5G?,» DIGI, 19 Marzo 2021. [En línea]. Available: https://es.digi.com/blog/post/5g-network-architecture.

[29] GSMA, «5G Implementation Guidelines: SA Option 2,» Junio 2020. [En línea]. Available: https://www.gsma.com/solutions-and-impact/technologies/networks/wp-content/uploads/2020/06/5G-SA-Option-2-ImplementationGuideline-v1.3.pdf.

[30] GSMA, «ROAD TO 5G: INTRODUCTION AND MIGRATION,» Abril 2018. [En línea]. Available: https://www.gsma.com/solutions-and-impact/technologies/networks/wp-content/uploads/2018/04/Road-to-5G-Introduction-and-Migration_FINAL.pdf.

[31] N. Singh, «What is the 5G Service-Based Architecture (SBA)?,» 26 Mayo 2023. [En línea]. Available: https://techcommunity.microsoft.com/t5/azure-for-operators-blog/what-is-the-5g-service-based-architecture-sba/ba-p/3831367.

[32] a. devbot5S, «DEVOPEDIA, 5G Service-Based Architecture,» 15 Febrero 2022. [En línea]. Available: https://devopedia.org/5g-service-based-architecture.

[33] ACL Digital, «Why Innovation in 5G Core Service-Based Architecture Matters,» 17 Julio 2023. [En línea]. Available: https://www.acldigital.com/blogs/why-innovation-5g-core-service-based-architecture-matters.

[34] J. I. M. L. M. C. M. A.-C. M. B. C. Jorge Andrés Brito, «Programmable Data Plane Applications in 5G and Beyond Architectures: A Systematic Review,» 4 Agosto 2023. [En línea]. Available: https://www.mdpi.com/1424-8220/23/15/6955.

[35] edgar-costa, «P4-Learning,» 9 Octubre 2023. [En línea]. Available: https://github.com/nsg-ethz/p4-learning/.

[36] erfanw, «QUIC Load Balancer Code,» 24 Septiembre 2020. [En línea]. Available: https://github.com/erfanw/QUIC_LB.

ANEXO A: CÓDIGOS COMPLETOS DE LOS ESCENARIOS

A.1 CÓDIGOS EL ESCENARIO 1

headers.p4

```
// HEADERS
const bit<16> TYPE_IPV4 = 0x800;

typedef bit<9>  egressSpec_t;
typedef bit<48> macAddr_t;
typedef bit<32> ip4Addr_t;


header ethernet_t {
    macAddr_t dstAddr;
    macAddr_t srcAddr;
    bit<16>   etherType;
}

header ipv4_t {
    bit<4>    version;
    bit<4>    ihl;
    bit<6>    dscp;
    bit<2>    ecn;
    bit<16>   totalLen;
    bit<16>   identification;
    bit<3>    flags;
    bit<13>   fragOffset;
    bit<8>    ttl;
    bit<8>    protocol;
    bit<16>   hdrChecksum;
    ip4Addr_t srcAddr;
    ip4Addr_t dstAddr;
}

header tcp_t{
    bit<16> srcPort;
    bit<16> dstPort;
    bit<32> seqNo;
    bit<32> ackNo;
    bit<4>  dataOffset;
    bit<4>  res;
    bit<1>  cwr;
    bit<1>  ece;
    bit<1>  urg;
    bit<1>  ack;
    bit<1>  psh;
    bit<1>  rst;
    bit<1>  syn;
```

```
    bit<1>  fin;
    bit<16> window;
    bit<16> checksum;
    bit<16> urgentPtr;
}

struct metadata {
    bit<14> ecmp_hash;
    bit<14> ecmp_group_id;
}

struct headers {
    ethernet_t   ethernet;
    ipv4_t       ipv4;
    tcp_t        tcp;
}
```

parsers.p4

```
// PARSER

parser MyParser(packet_in packet,
                out headers hdr,
                inout metadata meta,
                inout standard_metadata_t standard_metadata) {

    state start {
        transition parse_ethernet;
    }

    state parse_ethernet {
        packet.extract(hdr.ethernet);
        transition select(hdr.ethernet.etherType){
            TYPE_IPV4: parse_ipv4;
            default: accept;
        }
    }

    state parse_ipv4 {
        packet.extract(hdr.ipv4);
        transition select(hdr.ipv4.protocol){
            6 : parse_tcp;
            default: accept;
        }
    }

    state parse_tcp {
        packet.extract(hdr.tcp);
        transition accept;
    }
}

// DEPARSER
```

```
control MyDeparser(packet_out packet, in headers hdr) {
    apply {

        //parsed headers have to be added again into the packet.
        packet.emit(hdr.ethernet);
        packet.emit(hdr.ipv4);

        //Only emited if valid
        packet.emit(hdr.tcp);
    }
}
```

ecmp.p4

```
/* -*- P4_16 -*- */
#include <core.p4>
#include <v1model.p4>

#include "include/headers.p4"
#include "include/parsers.p4"

// CHECKSUM VERIFICATION

control MyVerifyChecksum(inout headers hdr, inout metadata meta) {
    apply {  }
}

// INGRESS PROECSSING

control MyIngress(inout headers hdr,
                  inout metadata meta,
                  inout standard_metadata_t standard_metadata) {
    action drop() {
        mark_to_drop(standard_metadata);
    }

    action ecmp_group(bit<14> ecmp_group_id, bit<16> num_nhops){
        hash(meta.ecmp_hash, HashAlgorithm.crc16, (bit<1>)0, { hdr.ipv4.srcAddr, hdr.ipv4.dstAddr,
                                            hdr.tcp.srcPort, hdr.tcp.dstPort, hdr.ipv4.protocol}, num_nhops);
        meta.ecmp_group_id = ecmp_group_id;
    }

    action set_nhop(macAddr_t dstAddr, egressSpec_t port) {
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
        hdr.ethernet.dstAddr = dstAddr;
        standard_metadata.egress_spec = port;
        hdr.ipv4.ttl = hdr.ipv4.ttl - 1;
    }



    table ecmp_group_to_nhop {
```

```
        key = {
            meta.ecmp_group_id:    exact;
            meta.ecmp_hash: exact;
        }
        actions = {
            drop;
            set_nhop;
        }
        size = 1024;
    }

    table ipv4_lpm {
        key = {
            hdr.ipv4.dstAddr: lpm;
        }
        actions = {
            set_nhop;
            ecmp_group;
            drop;
        }
        size = 1024;
        default_action = drop;
    }

    apply {
        if (hdr.ipv4.isValid()){
            switch (ipv4_lpm.apply().action_run){
                ecmp_group: {
                    ecmp_group_to_nhop.apply();
                }
            }
        }
    }
}

// EGRESS PROCESSING

control MyEgress(inout headers hdr,
                 inout metadata meta,
                 inout standard_metadata_t standard_metadata) {
    apply {

    }
}

// CHECKSUM COMPUTATION

control MyComputeChecksum(inout headers hdr, inout metadata meta) {
     apply {
        update_checksum(
        hdr.ipv4.isValid(),
            { hdr.ipv4.version,
              hdr.ipv4.ihl,
              hdr.ipv4.dscp,
              hdr.ipv4.ecn,
```

```
            hdr.ipv4.totalLen,
            hdr.ipv4.identification,
            hdr.ipv4.flags,
            hdr.ipv4.fragOffset,
            hdr.ipv4.ttl,
            hdr.ipv4.protocol,
            hdr.ipv4.srcAddr,
            hdr.ipv4.dstAddr },
            hdr.ipv4.hdrChecksum,
            HashAlgorithm.csum16);
    }
}

// SWITCH

V1Switch(
MyParser(),
MyVerifyChecksum(),
MyIngress(),
MyEgress(),
MyComputeChecksum(),
MyDeparser()
) main;
```

s1-commands.txt

```
//SWITCH principal conectado a h1

table_set_default ipv4_lpm drop
table_set_default ecmp_group_to_nhop drop

table_add ipv4_lpm set_nhop 10.0.1.1/32 =>  00:00:0a:00:01:01 1
table_add ipv4_lpm ecmp_group 10.0.6.2/32 => 1 4

//ecmp id:1 port 0,1,2,3
table_add ecmp_group_to_nhop set_nhop 1 0 =>  00:00:00:02:01:00 2
table_add ecmp_group_to_nhop set_nhop 1 1 =>  00:00:00:03:01:00 3
table_add ecmp_group_to_nhop set_nhop 1 2 =>  00:00:00:04:01:00 4
table_add ecmp_group_to_nhop set_nhop 1 3 =>  00:00:00:05:01:00 5
```

sX-commands.txt para sw intermedios (X=2, 3, 4 y 5)

```
//SWITCHES secundarios, conectados a s1 y s2

table_set_default ipv4_lpm drop
table_set_default ecmp_group_to_nhop drop

table_add ipv4_lpm set_nhop 10.0.1.1/32 => 00:00:00:01:03:00 1
table_add ipv4_lpm set_nhop 10.0.6.2/32 => 00:00:00:06:03:00 2
```

s6-commands.txt

```
//SWITCH principal conectado a h2

table_set_default ipv4_lpm drop
table_set_default ecmp_group_to_nhop drop

table_add ipv4_lpm set_nhop 10.0.6.2/32 =>  00:00:0a:00:06:02 1
table_add ipv4_lpm ecmp_group 10.0.1.0/24 => 1 4

//ecmp id:1 port 0,1,2,3
table_add ecmp_group_to_nhop set_nhop 1 0 =>  00:00:00:02:06:00 2
table_add ecmp_group_to_nhop set_nhop 1 1 =>  00:00:00:03:06:00 3
table_add ecmp_group_to_nhop set_nhop 1 2 =>  00:00:00:04:06:00 4
table_add ecmp_group_to_nhop set_nhop 1 3 =>  00:00:00:05:06:00 5
```

network.py

```
from p4utils.mininetlib.network_API import NetworkAPI

net = NetworkAPI()
net.setLogLevel('info')

# Network definition
net.addP4Switch('s1', cli_input='sw-commands/s1-commands.txt')
net.addP4Switch('s2', cli_input='sw-commands/s2-commands.txt')
net.addP4Switch('s3', cli_input='sw-commands/s3-commands.txt')
net.addP4Switch('s4', cli_input='sw-commands/s4-commands.txt')
net.addP4Switch('s5', cli_input='sw-commands/s5-commands.txt')
net.addP4Switch('s6', cli_input='sw-commands/s6-commands.txt')
net.setP4SourceAll('p4src/ecmp.p4')

net.addHost('h1')
net.addHost('h2')
net.addLink("h1", "s1")
net.addLink("h2", "s6")
net.addLink("s1", "s2")
net.addLink("s1", "s3")
net.addLink("s1", "s4")
net.addLink("s1", "s5")
net.addLink("s2", "s6")
net.addLink("s3", "s6")
net.addLink("s4", "s6")
net.addLink("s5", "s6")

# Assignment strategy
net.mixed()

# Nodes general options
net.enablePcapDumpAll()
net.enableLogAll()
net.enableCli()
net.startNetwork()
```

p4app.json

```
{
  "p4_src": "p4src/ecmp.p4",
  "cli": true,
  "pcap_dump": true,
  "enable_log": true,
  "topology": {
    "assignment_strategy": "mixed",
    "links": [["h1", "s1"], ["h2", "s6"], ["s1", "s2"], ["s1", "s3"], ["s1", "s4"], ["s1", "s5"], ["s2", "s6"],
["s3", "s6"], ["s4", "s6"], ["s5", "s6"]],
    "hosts": {
      "h1": {
      },
      "h2": {
      }
    },
    "switches": {
      "s1": {
        "cli_input": "sw-commands/s1-commands.txt"
      },
      "s2": {
        "cli_input": "sw-commands/s2-commands.txt"
      },
      "s3": {
        "cli_input": "sw-commands/s3-commands.txt"
      },
      "s4": {
        "cli_input": "sw-commands/s4-commands.txt"
      },
      "s5": {
        "cli_input": "sw-commands/s5-commands.txt"
      },
      "s6": {
        "cli_input": "sw-commands/s6-commands.txt"
      }
    }
  }
}
```

send.py

```
#!/usr/bin/env python3
import argparse
import sys
import socket
import random
import struct

from scapy.all import sendp, get_if_list, get_if_hwaddr
from scapy.all import Ether, IP, UDP, TCP

def get_if():
```

```
    ifs=get_if_list()
    iface=None # "h1-eth0"
    for i in get_if_list():
        if "eth0" in i:
            iface=i
            break
    if not iface:
        print("Cannot find eth0 interface")
        exit(1)
    return iface

def main():

    if len(sys.argv)<3:
        print('pass 2 arguments: <destination> <number_of_random_packets>')
        exit(1)

    addr = socket.gethostbyname(sys.argv[1])
    iface = get_if()

    print("sending on interface %s to %s" % (iface, str(addr)))

    for _ in range(int(sys.argv[2])):
        pkt = Ether(src=get_if_hwaddr(iface), dst='ff:ff:ff:ff:ff:ff')
        pkt = pkt /IP(dst=addr) / TCP(dport=random.randint(5000,60000),
sport=random.randint(49152,65535))
        sendp(pkt, iface=iface, verbose=False)

if __name__ == '__main__':
    main()
```

A.2 CÓDIGOS EL ESCENARIO 2

headers.p4

```
// HEADERS

const bit<16> TYPE_IPV4 = 0x0800;

typedef bit<9>  egressSpec_t;
typedef bit<48> macAddr_t;
typedef bit<32> ip4Addr_t;
typedef bit<64> connID_t;

header ethernet_t {
    macAddr_t dstAddr;
    macAddr_t srcAddr;
    bit<16>   etherType;
}


header ipv4_t {
```

```
    bit<4>    version;
    bit<4>    ihl;
    bit<6>    dscp;
    bit<2>    ecn;
    bit<16>   totalLen;
    bit<16>   identification;
    bit<3>    flags;
    bit<13>   fragOffset;
    bit<8>    ttl;
    bit<8>    protocol;
    bit<16>   hdrChecksum;
    ip4Addr_t srcAddr;
    ip4Addr_t dstAddr;
}
header udpQuic_t {
    bit<16> srcPort;
    bit<16> dstPort;
    bit<16> length;
    bit<16> checksum;
    bit<1> hdr_type;
    bit<1> fixed;
    bit<2> pkt_type;
    bit<4> version;
}
header quicLong_t{
    bit<32> version;
    bit<8> dcid_length;
    bit<8> dcid_first_byte;
    bit<16> cookie;
    bit<40> dcid_residue;
    bit<8> scid_length;
    bit<64> src_cid;
}
header quicShort_t{
    bit<8> dcid_first_byte;
    bit<16> cookie;
    bit<40> dcid_residue;
}

struct metadata {
   bit<16> bucket_id;
   bit<16> hash;
   bit<16> crc16;
   bit<16> server_id;
}
struct headers {
    ethernet_t   ethernet;
    ipv4_t ipv4;
    udpQuic_t udpQuic;
    quicShort_t quicShort;
    quicLong_t quicLong;
}
```

parsers.p4

```
// PARSER

parser MyParser(packet_in packet, out headers hdr, inout metadata meta, inout standard_metadata_t
                                                                                    standard_metadata)
{
    state start {
        transition parse_ethernet;
    }
    state parse_ethernet {
        packet.extract(hdr.ethernet);
        transition select(hdr.ethernet.etherType){
            TYPE_IPV4: parse_ipv4;
            default: accept;
        }
    }
    state parse_ipv4 {
        packet.extract(hdr.ipv4);
        transition select(hdr.ipv4.protocol){
            17: parse_udpQuic;
            default: accept;
        }
    }
    state parse_udpQuic {
        packet.extract(hdr.udpQuic);
        transition select(hdr.udpQuic.hdr_type){
            0: parse_quicShort;
            1: parse_quicLong;
            default: accept;
        }
    }
    state parse_quicShort {
        packet.extract(hdr.quicShort);
        transition accept;
    }
    state parse_quicLong {
        packet.extract(hdr.quicLong);
        transition accept;
    }
}

// DEPARSER

control MyDeparser(packet_out packet, in headers hdr) {
    apply {
        packet.emit(hdr.ethernet);
        packet.emit(hdr.ipv4);
        packet.emit(hdr.udpQuic);
        packet.emit(hdr.quicShort);
        packet.emit(hdr.quicLong);
    }
}
```

l2_basic_forwarding.p4

```
/* -*- P4_16 -*- */
#include <core.p4>
#include <v1model.p4>
#include "include/parsers.p4"

// HEADERS

#include "include/headers.p4"
#define BUCKET_SIZE 6
#define COUNTER_WIDTH 16

// PARSERS
#include "include/parsers.p4"

// CHECKSUM VERIFICATION

control MyVerifyChecksum(inout headers hdr, inout metadata meta) {
    apply {  }
}

// INGRESS PROCESSING

control MyIngress(inout headers hdr, inout metadata meta, inout standard_metadata_t
                                                                    standard_metadata) {
    bit<32> virtual_ip = 0x0a0000fe;
    bit<48> client_mac = 0x00000a000001;

    //Counter for the WRR
    register<bit<COUNTER_WIDTH>>(1) bucket_counter;

    //A register for debugging, internal use only
    //register<bit<16>>(1) debug_hash;

    action crc_hash(bit<16> port, bit<32> addr){
        hash(meta.crc16, HashAlgorithm.crc16, (bit<16>)0, {port, addr}, (bit<16>)65535);
    }

    action fwd_to_server(bit<9> egress_port, bit<32> dip, bit<48> dmac) {
        hdr.ipv4.dstAddr = dip;
        hdr.ethernet.dstAddr = dmac;
        standard_metadata.egress_spec = egress_port;
    }

    table get_server_from_bucket {
        key = {
            meta.bucket_id: exact;
        }
        actions = {
            fwd_to_server;
        }
        size = 1024;
    }
```

```
    table get_server_from_id_long_header {
        key = {
            meta.server_id: exact;
        }
        actions = {
            fwd_to_server;
        }
        size = 1024;
    }

    table get_server_from_id_short_header {
        key = {
            meta.server_id: exact;
        }
        actions = {
            fwd_to_server;
        }
        size = 1024;
    }

        // Incremental checksum fix adapted from the pseudocode at https://p4.org/p4-spec/docs/PSA-
v1.1.0.html#appendix-internetchecksum-implementation
    action ones_complement_sum(in bit<16> x, in bit<16> y, out bit<16> sum) {
        bit<17> ret = (bit<17>) x + (bit<17>) y;
        if (ret[16:16] == 1) {
            ret = ret + 1;
        }
        sum = ret[15:0];
    }

    // Restriction: data is a multiple of 16 bits long
    action subtract(inout bit<16> sum, bit<16> d) {
            ones_complement_sum(sum, ~d, sum);
    }

    action subtract32(inout bit<16> sum, bit<32> d) {
            ones_complement_sum(sum, ~(bit<16>)d[15:0], sum);
            ones_complement_sum(sum, ~(bit<16>)d[31:16], sum);
    }

    action add(inout bit<16> sum, bit<16> d) {
            ones_complement_sum(sum, d, sum);
    }

    action add32(inout bit<16> sum, bit<32> d) {
            ones_complement_sum(sum, (bit<16>)(d[15:0]), sum);
            ones_complement_sum(sum, (bit<16>)(d[31:16]), sum);
    }


    apply {
        bit <16> sum = 0;
        subtract(sum, hdr.udpQuic.checksum);
        subtract32(sum, hdr.ipv4.dstAddr);
        subtract32(sum, hdr.ipv4.srcAddr);
```

```
        subtract(sum, hdr.udpQuic.length);

        if(hdr.udpQuic.isValid()){              //Only process Quic packets
            if(hdr.ipv4.dstAddr == virtual_ip) {    //packet from client
                if((hdr.udpQuic.hdr_type == 1) && (hdr.udpQuic.pkt_type == (bit<2>)0)){       //Initial,
must be long header
                    bucket_counter.read(meta.bucket_id, 0);

                    // we use the bucket index to find the server
                    get_server_from_bucket.apply();

                    // new connection, update counter
                    meta.bucket_id = meta.bucket_id + 1;

                    //Do the wrapping
                    if (meta.bucket_id == BUCKET_SIZE) {
                        meta.bucket_id = 0;
                    }
                    bucket_counter.write(0, meta.bucket_id);
                }
                else {                          //non-Initial, the dcid must have a server_id.
                    if (hdr.udpQuic.hdr_type == 1){
                        crc_hash(hdr.udpQuic.srcPort, hdr.ipv4.srcAddr);
                        //debug_hash.write(0, meta.crc16);
                        meta.server_id = meta.crc16 ^ hdr.quicLong.cookie;
                        get_server_from_id_long_header.apply();
                    }
                    else{
                        crc_hash(hdr.udpQuic.srcPort, hdr.ipv4.srcAddr);
                        meta.server_id = meta.crc16 ^ hdr.quicShort.cookie;
                        get_server_from_id_short_header.apply();
                    }
                }
            }

            else {                              //packet from servers. Just forward to the client.
                hdr.ipv4.srcAddr = virtual_ip;
                hdr.ethernet.dstAddr = client_mac;
                standard_metadata.egress_spec = (bit<9>)1;
            }
        }


        add(sum, hdr.udpQuic.length);
        add32(sum, hdr.ipv4.srcAddr);
        add32(sum, hdr.ipv4.dstAddr);

        hdr.udpQuic.checksum = ~sum;

    }

}


//EGRESS PROCESSING
```

```
control MyEgress(inout headers hdr, inout metadata meta, inout standard_metadata_t
                                                                    standard_metadata) {
    apply {  }
}

// CHECKSUM COMPUTATION

control MyComputeChecksum(inout headers hdr, inout metadata meta) {
   // bit<8> eight_zeroes = (bit<8>)0;
    apply {
        update_checksum(
            hdr.ipv4.isValid(),
                {
                    hdr.ipv4.version,
                    hdr.ipv4.ihl,
                    hdr.ipv4.dscp,
                    hdr.ipv4.ecn,
                    hdr.ipv4.totalLen,
                    hdr.ipv4.identification,
                    hdr.ipv4.flags,
                    hdr.ipv4.fragOffset,
                    hdr.ipv4.ttl,
                    hdr.ipv4.protocol,
                    hdr.ipv4.srcAddr,
                    hdr.ipv4.dstAddr
                },
                hdr.ipv4.hdrChecksum,
                HashAlgorithm.csum16
        );
    }
}

// SWITCH

//switch architecture
V1Switch(
MyParser(),
MyVerifyChecksum(),
MyIngress(),
MyEgress(),
MyComputeChecksum(),
MyDeparser()
) main;
```

Makefile

```
server1:
    cd picoquic && ./picoquic_sample server 4443 ./certs/cert.pem ./certs/key.pem ./server_files 1
server2:
    cd picoquic && ./picoquic_sample server 4443 ./certs/cert.pem ./certs/key.pem ./server_files 2
client:
    arp -s 10.0.0.254 a1:23:e3:8d:4b:01 && cd ../picoquic && ./picoquic_sample client 10.0.0.254
        4443 ./client_files index.html
```

s1-command.txt

```
table_add MyIngress.get_server_from_bucket MyIngress.fwd_to_server 0 => 2 10.0.0.2
00:00:0a:00:00:02
table_add MyIngress.get_server_from_bucket MyIngress.fwd_to_server 1 => 2 10.0.0.2
00:00:0a:00:00:02
table_add MyIngress.get_server_from_bucket MyIngress.fwd_to_server 2 => 2 10.0.0.2
00:00:0a:00:00:02
table_add MyIngress.get_server_from_bucket MyIngress.fwd_to_server 3 => 2 10.0.0.2
00:00:0a:00:00:02
table_add MyIngress.get_server_from_bucket MyIngress.fwd_to_server 4 => 3 10.0.0.3
00:00:0a:00:00:03
table_add MyIngress.get_server_from_bucket MyIngress.fwd_to_server 5 => 3 10.0.0.3
00:00:0a:00:00:03

table_add MyIngress.get_server_from_id_short_header MyIngress.fwd_to_server 1 => 2 10.0.0.2
00:00:0a:00:00:02
table_add MyIngress.get_server_from_id_short_header MyIngress.fwd_to_server 2 => 3 10.0.0.3
00:00:0a:00:00:03

table_add MyIngress.get_server_from_id_long_header MyIngress.fwd_to_server 1 => 2 10.0.0.2
00:00:0a:00:00:02
table_add MyIngress.get_server_from_id_long_header MyIngress.fwd_to_server 2 => 3 10.0.0.3
00:00:0a:00:00:03

register_write MyIngress.bucket_counter 0 0
```

crc16.c

```
#include <stdint.h>
#include <stdio.h>

#define CRC16 0x8005

uint16_t gen_crc16(const uint8_t *data, uint16_t size)
{
    uint16_t out = 0;
    int bits_read = 0, bit_flag;

    /* Sanity check: */
    if(data == NULL)
        return 0;
```

```
    while(size > 0)
    {
        bit_flag = out >> 15;

        /* Get next bit: */
        out <<= 1;
        out |= (*data >> bits_read) & 1; // item a) work from the least significant bits

        /* Increment bit counter: */
        bits_read++;
        if(bits_read > 7)
        {
            bits_read = 0;
            data++;
            size--;
        }

        /* Cycle check: */
        if(bit_flag)
            out ^= CRC16;

    }

    // item b) "push out" the last 16 bits
    int i;
    for (i = 0; i < 16; ++i) {
        bit_flag = out >> 15;
        out <<= 1;
        if(bit_flag)
            out ^= CRC16;
    }

    // item c) reverse the bits
    uint16_t crc = 0;
    i = 0x8000;
    int j = 0x0001;
    for (; i != 0; i >>=1, j <<= 1) {
        if (i & out) crc |= j;
    }

    return crc;
}

int main(){
    char num[] = "12345678";
    uint16_t size = sizeof(num) - 1;
    uint16_t crc16 =  gen_crc16(num, size);
    printf("%x\n", crc16);

    return 0;
}
```

p4app.json

```
{
  "p4_src": "p4src/l2_basic_forwarding.p4",
  "switch": "simple_switch",
  "compiler": "p4c",
  "options": "--target bmv2 --arch v1model --std p4-16",
  "switch_cli": "simple_switch_CLI",
  "cli": true,
  "pcap_dump": true,
  "enable_log": true,
  "topo_module": {
    "file_path": "",
    "module_name": "p4utils.mininetlib.apptopo",
    "object_name": "AppTopoStrategies"
  },
  "controller_module": null,
  "topodb_module": {
    "file_path": "",
    "module_name": "p4utils.utils.topology",
    "object_name": "Topology"
  },
  "mininet_module": {
    "file_path": "",
    "module_name": "p4utils.mininetlib.net",
    "object_name": "P4Mininet"
  },
  "topology": {
    "assignment_strategy": "l2",
    "links": [["h1", "s1"], ["h2", "s1"], ["h3", "s1"]],
    "hosts": {
      "h1": {
      },
      "h2": {
      }
      ,
      "h3": {
      }

    },
    "switches": {
      "s1": {
        "cli_input": "s1-commands.txt"
      }
    }
  }
}
```

ANEXO B: ASPECTOS ÉTICOS, ECONÓMICOS, SOCIALES Y AMBIENTALES

En este anexo se analizan los impactos éticos, económicos, sociales y ambientales relacionados con este proyecto, de acuerdo con la normativa de TFGs de la ETSIT-UPM.

B.1 INTRODUCCIÓN

Como hemos comprobado, el objetivo principal de este proyecto es el análisis de las tecnologías de los planos de datos programables (y los posibles protocolos empleados en ellas) para implementarlos en aplicaciones de redes 5G. Estas redes y por ende, las tecnologías que emplean están teniendo un veloz desarrollo, por lo que la implementación de soluciones veloces, eficaces y flexibles otorga un gran impacto positivo en todos los aspectos que destaca este anexo.

B.2 DESCRIPCIÓN DE IMPACTOS RELEVANTES RELACIONADOS CON EL PROYECTO

La investigación de mejoras en las tecnologías, sobre todo viendo que puede mejorar aspectos vitales importantes como son la economía, la sociedad o el medio ambiente, es nuestro deber ético como ingenieros, por lo que resulta crucial sostener las investigaciones y desarrollos plasmados para un futuro mejor.

La actualización de las redes y la inversión en el estudio e implementación de estas tecnologías puede causar una inversión económica grande, pero evaluando la posibilidad de reutilización de los nuevos elementos reconfigurables, se prevé una amortización del gasto inicial de muchos años, mejorando a largo plazo las inversiones anuales en la tecnología, además de la mejora que producirían estos avances.

La implementación de todas estas herramientas en las aplicaciones 5G venideras generan grandes ventajas sociales para todo el mundo ya que mejorarían la calidad del uso de las tecnologías actuales para la sociedad, facilitando a las personas muchas acciones cotidianas al emplear dispositivos inteligentes y mejorando la experiencia de usuario del uso de la red.

La generación de residuos obsoletos es uno de los mayores problemas a resolver en la industria tecnológica. La implementación de estas tecnologías mencionadas genera una vida de uso de los dispositivos mucho mayor al conseguir su reprogramabilidad y la virtualización de muchos elementos, por lo que la disminución de estos residuos tecnológicos es notoria.

B.3 ANÁLISIS DETALLADO DE ALGUNO DE LOS PRINCIPALES IMPACTOS

Teniendo en cuenta el largo plazo, los planos de los impactos económico y ambiental pueden sobresalir sobre demás. Esto se debe por la propia naturaleza de los contenidos plasmados en el trabajo, ya que trata de dispositivos reutilizables, reconfigurables, de libre uso personal, virtuales, eficientes y flexibles.

Esto generaría una menor inversión de equipos físicos, generando un gran impacto en el plano económico, y a su vez, una menor contaminación de recursos materiales empleados, mejorando el aspecto medioambiental, siendo un proceso tecnológico más renovable y sostenible.

B.4 CONCLUSIONES

A lo largo del presente trabajo se han comentado herramientas y necesidades que requiere el nuevo entendimiento de las redes, como el dinamismo o la flexibilidad. Dichos requisitos tienen un impacto positivo hacia el medioambiente, ya que permiten la reutilización de los elementos de red consiguiendo así mejorar la utilización de recursos. Asimismo, el campo económico también obtiene grandes beneficios, ya que al poder utilizar los equipos en varios escenarios se requiere menos equipos específicos. Además de esto, esto mejoraría las tecnologías empleadas por el público general generando un impacto social positivo. Por estas razones, éticamente es nuestra labor aportar al progreso de la investigación y desarrollo de estas herramientas para lograr los objetivos buscados.

ANEXO C: PRESUPUESTO ECONÓMICO

COSTE DE MANO DE OBRA (coste directo)	**Horas**	**Precio/hora**	**Total**
	350	15 €	**5.250 €**

COSTE DE RECURSOS MATERIALES (coste directo)	**Precio de compra**	**Uso en meses**	**Amortización (en años)**	**Total**
Ordenador personal (Software y periféricos incluidos)	900,00 €	7	5	105 €
COSTE TOTAL				**105 €**

GASTOS GENERALES (costes indirectos)	15%	sobre CD	**803,25 €**
BENEFICIO INDUSTRIAL	6%	sobre CD+CI	**369,50 €**

SUBTOTAL PRESUPUESTO		**5.355,00 €**
IVA APLICABLE	21%	**1.124,55 €**

TOTAL PRESUPUESTO	**6.479,55 €**